Building the compacted colored de Bruijn Graph

Construction of the compacted colored De Bruijn Graph from *reference* sequence

Bioinformatics, 33(24), 2017, 4024–4032
doi: 10.1093/bioinformatics/btw609
Advance Access Publication Date: 21 September 2016
Original Paper

OXFORD

Sequence analysis

TwoPaCo: an efficient algorithm to build the compacted de Bruijn graph from many complete genomes

Ilia Minkin[1], Son Pham[2] and Paul Medvedev[1,3,4,*]

[1]Department of Computer Science and Engineering, The Pennsylvania State University, University Park, PA 16802, USA, [2]BioTuring Inc., San Diego, CA 92121, USA, [3]Department of Biochemistry and Molecular Biology and [4]Genomic Sciences Institute of the Huck, The Pennsylvania State University, University Park, PA 16802, USA

*To whom correspondence should be addressed.

Associate Editor: Alfonso Valencia

Received on April 3, 2016; revised on September 1, 2016; accepted on September 16, 2016

TwoPaCo: An efficient algorithm to build the compacted de Bruijn graph from many complete genomes

Ilia Minkin[1], Son Pham[2], Paul Medvedev[1]

Pennsylvania State University[1]
Salk Institute for Biological Studies[2]

8th July 2016

TwoPaCo slides, unless otherwise noted, are from Ilia's presentation

Motivation

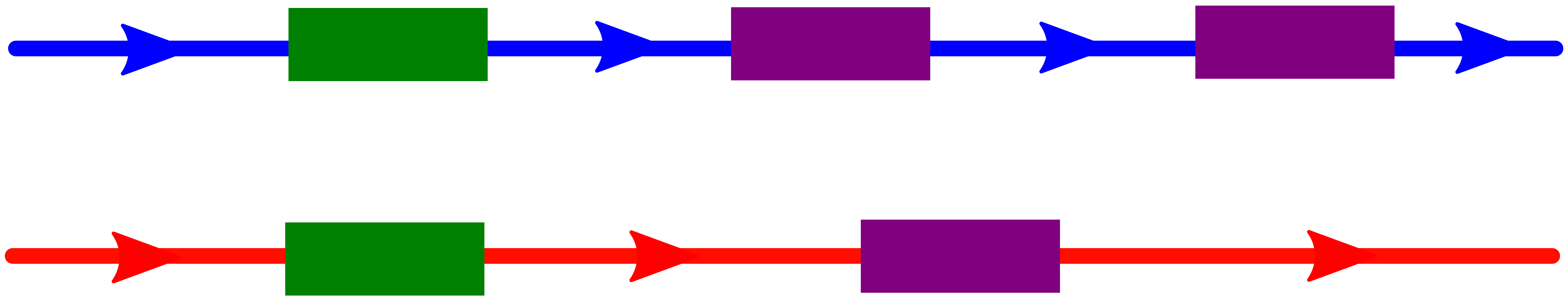
- More and more complete genomes
- Pan-genome: analysis within same species
- Mammalian-sized genomes are common & larger genomes are becoming so

Key question: what is a handy data structure to represent genomes?

The simplest way: string(s) of characters.

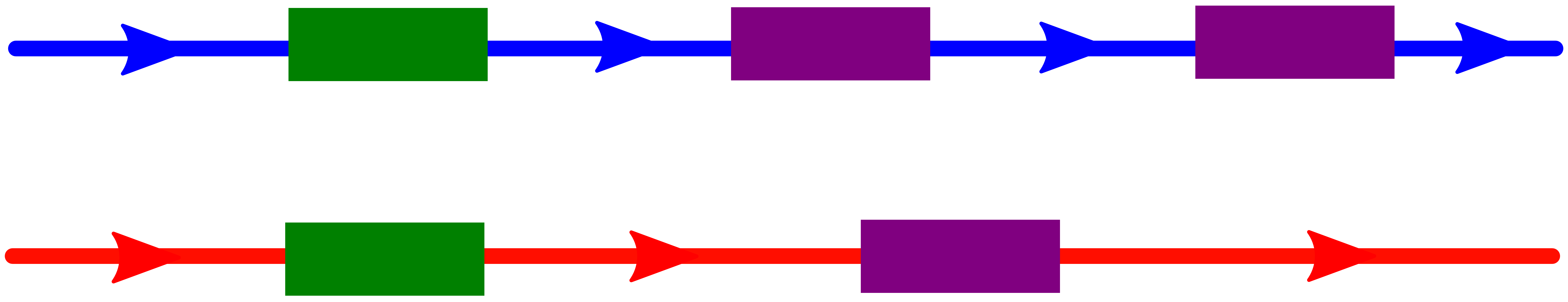
The Linear Representation

Two genomes:



The Linear Representation

Two genomes:

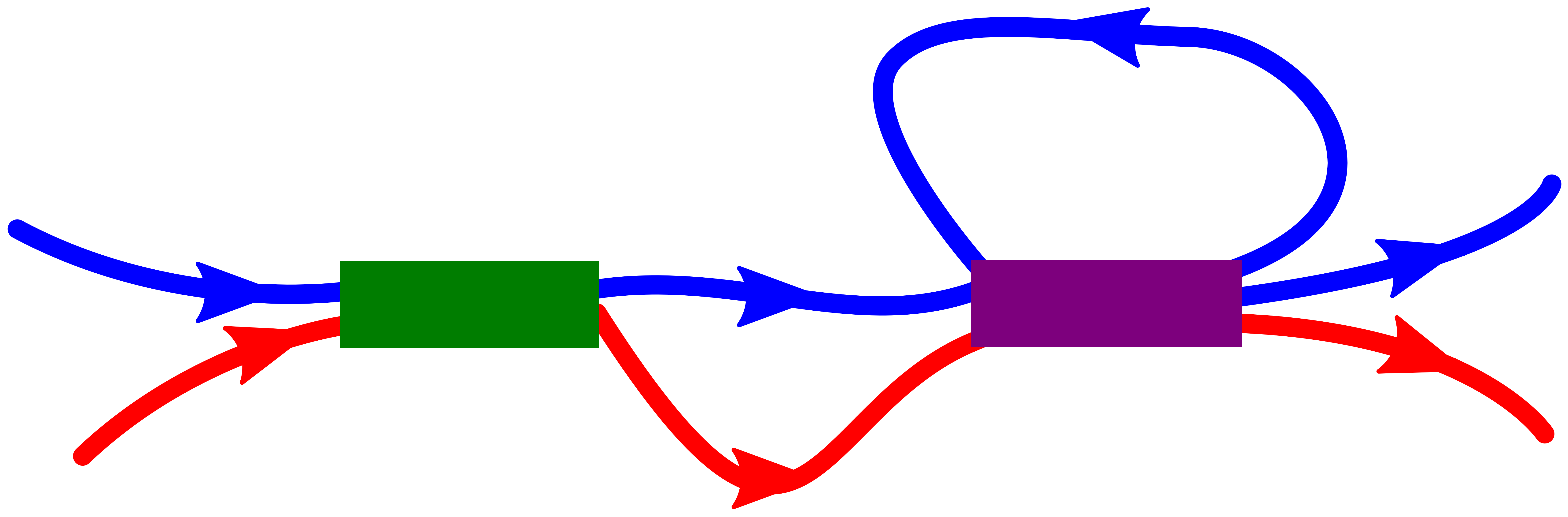


Issues:

- Homology between genomes?
- Duplications?
- Rearrangements?

Solution: a Graph Representation

What we want to see:



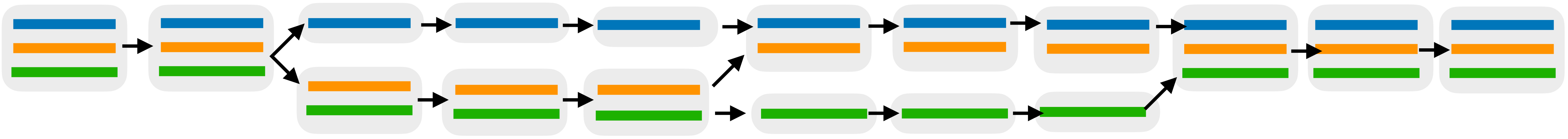
Constructing the DBG efficiently

When the input are reference sequences (i.e. genomes)

Consider a collection of genomes; each given a “color”

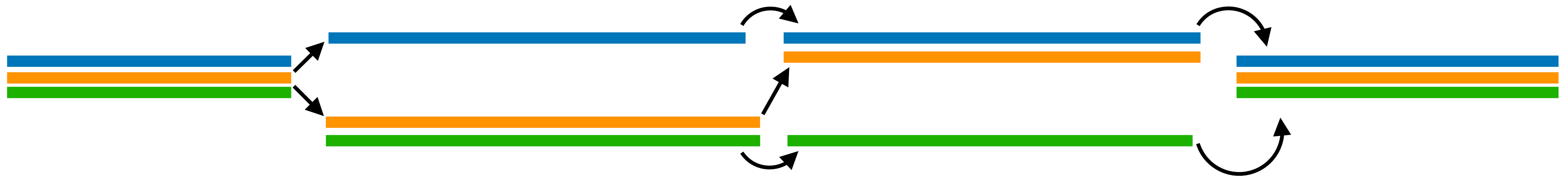
Build a DBG from the k-mer set of the collection.
Each k-mer is associated with a “color set”.

Shared sequence → shared sub-paths in the graph



The “raw” DBG is large & memory inefficient.
We want the ***<u>compacted</u>*** graph.

Every non-branching path is collapsed into a single vertex, labeled with the string its subpath spells.



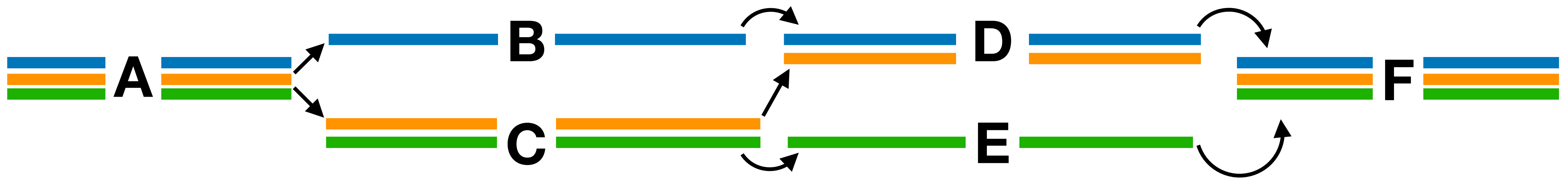
Aside: The DBG as a “factorization” of the genomes

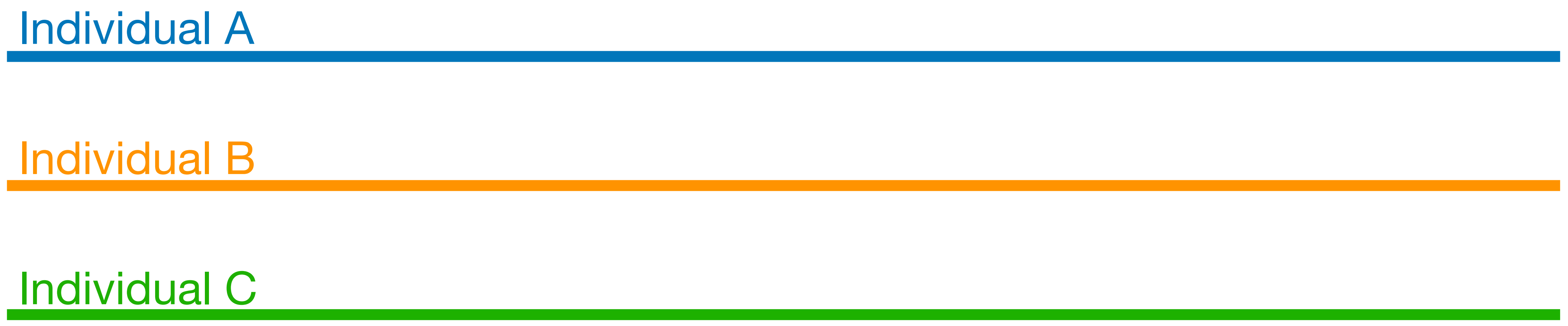
Efficient representation & compression through “factorization”

$$(2 \times 4) + (2 \times 5) + (3 \times 8) + (2 \times 6) + (3 \times 4) + (3 \times 5) = 2\,(4+5+6) + 3\,(8+4+5)$$

$$\downarrow$$

- Redundant sequences (repeats) are implicitly collapsed.
- Both *within* and *across* genomes.





= A,B,D,F

= A,C,D,F

= A,C,E,F

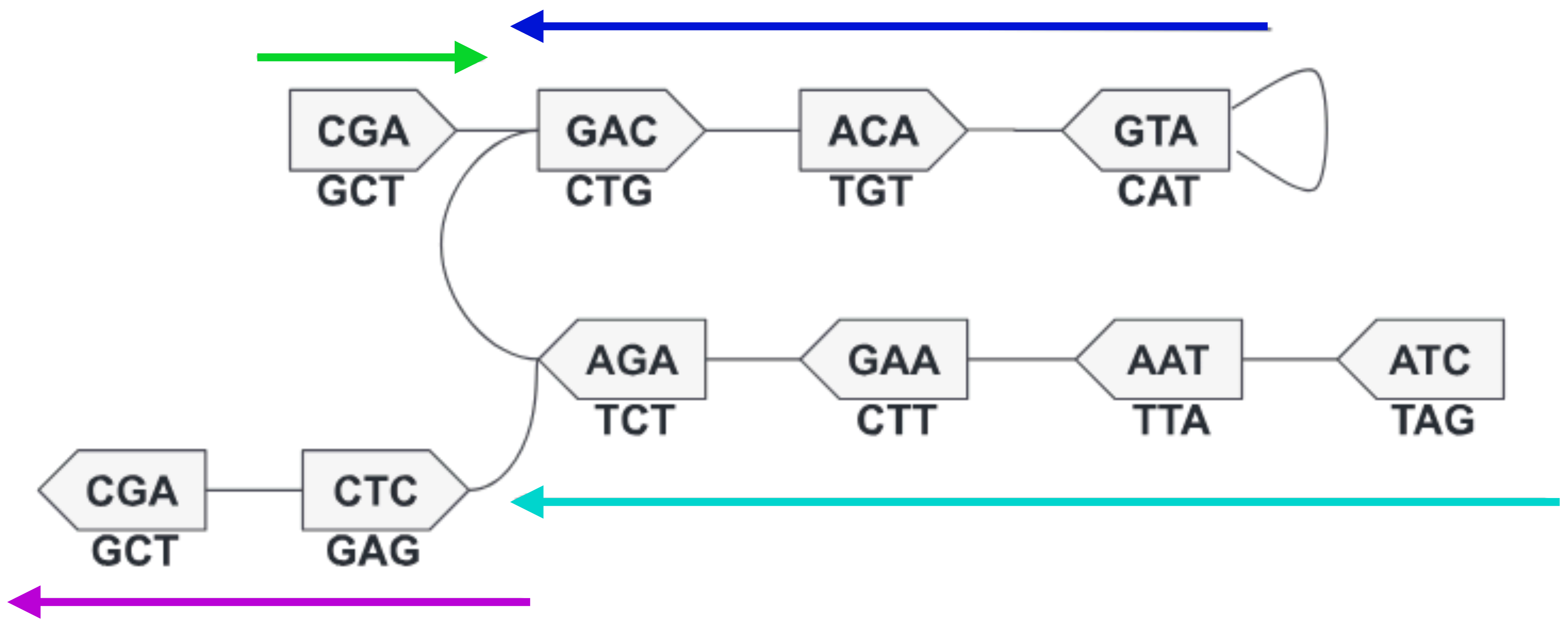
- The “unitigs” (non—branching paths) of the De Bruijn graph of order k can be seen to factorize a collection of sequences into its constituent set of maximal substrings such that

1. Each k-mer appears in exactly 1 such substring

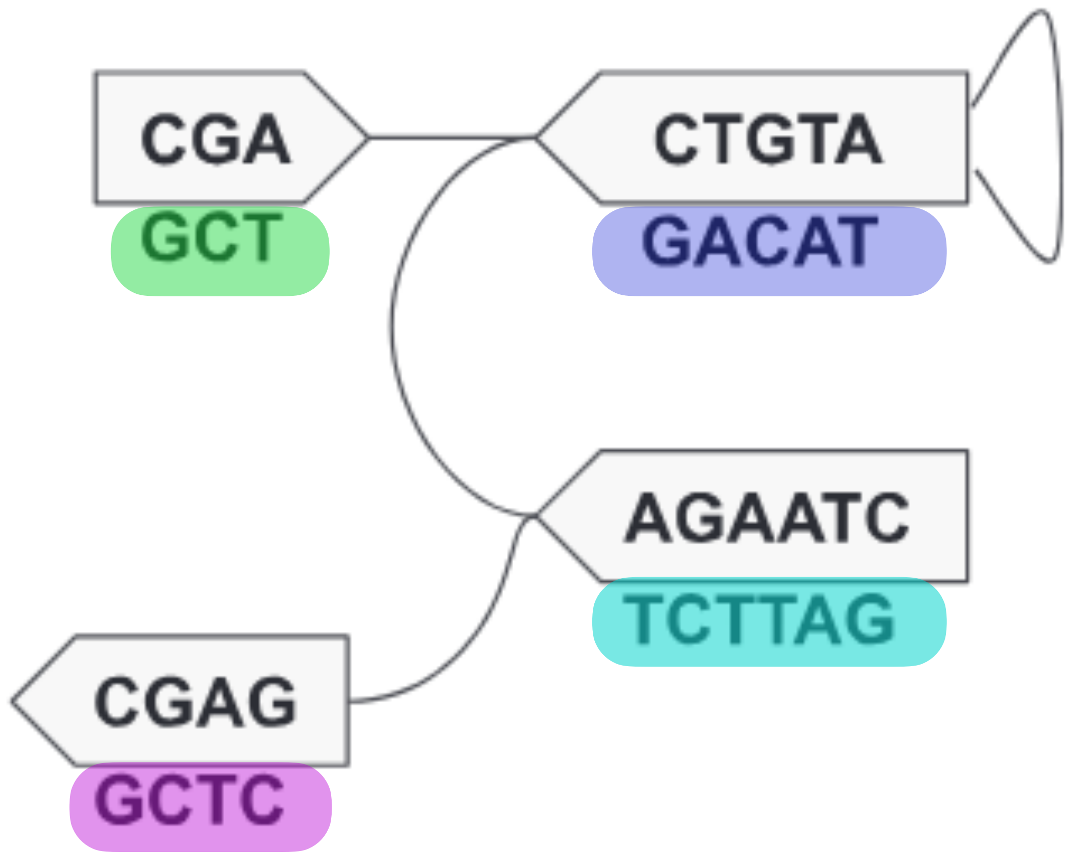
2. Each string s in the set can be spelled out as a sequence of these substrings.

Constructing the DBG efficiently

We consider here a carefully constructed *bi-directed* graph, and note that our formulation properly handles the tricky case that a node in the DBG encodes both a k-mer and its *reverse-complement.* We elide the details of reverse-complement handling in this presentation for our own sanity*.



$G(S, k)$ **for** $S = \{\text{CGACATGTCTTAG}, \text{GCTCTTAG}\}$ **with** $k = 3$



compacted form of $G(S, k)$, $G_c(S, k)$

> * This case forms an exception to the rule that all edges come in pairs with their mirror.
> I suspect that this special case has cost humanity thousand of hours in debugging time.
> — Rayan Chikhi (CNRS)

Cuttlefish slides adapted from slides from Jamshed K.

Constructing the DBG efficiently

How *not* to do it

Given a set of references S

```
Naïve-Compaction(S):
  1. G = Construct-de-Bruijn-Graph(S)
  2. G_c = Compact-Using-Linear-Traversal(S)
  3. return G_c
```

Extracting & compacting non-branching paths is ***conceptually*** trivial (e.g. via DFS/BFS)

In practice: Totally infeasible — space requirements are enormous $|G_c| \ll |G|$

Need to bypass explicit G(R, k) construction; *must output compacted paths directly*

Why?

A path of L vertices takes space $k \times L$ in the uncompacted graph, but $k + L - 1$ in the compacted graph

Constructing the DBG efficiently

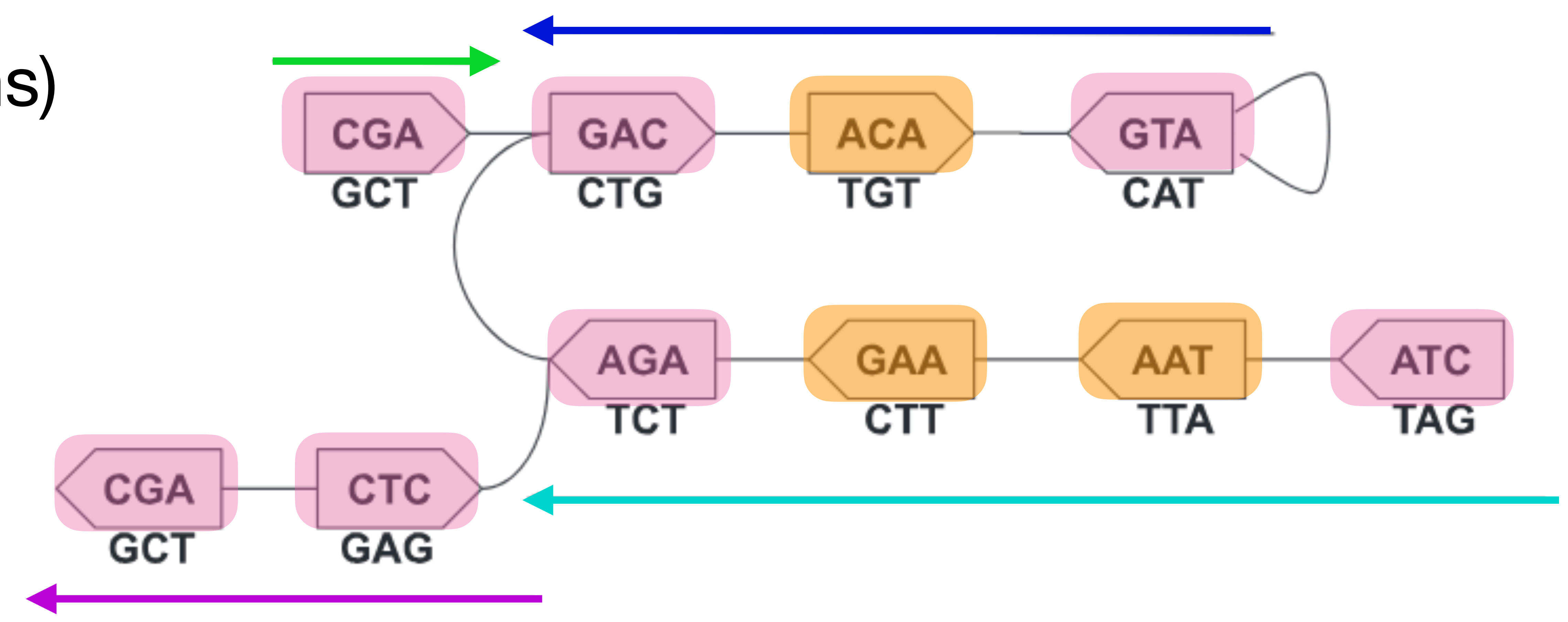
How to do it

Idea: Implicit *walk* over G(S, k)

For an *edge-centric* DBG, a complete walk traversal w(s) over G(s, k) can be obtained via a scan over s, *without having G(s, k) itself present in memory*

and the maximal unitigs (i.e. maximal non-branching paths) of G(s, k) are contained as subpaths in this walk w(s)

If each $v \in G(S, k)$ can be characterized as "flanking" or "internal" (w.r.t each maximal unitig), the unitig sequences themselves can be extracted by identifying subpaths having flanking vertices at both ends.



CGA, GAC, ATG, AGA, CTA, AGC, and CTC are flanking

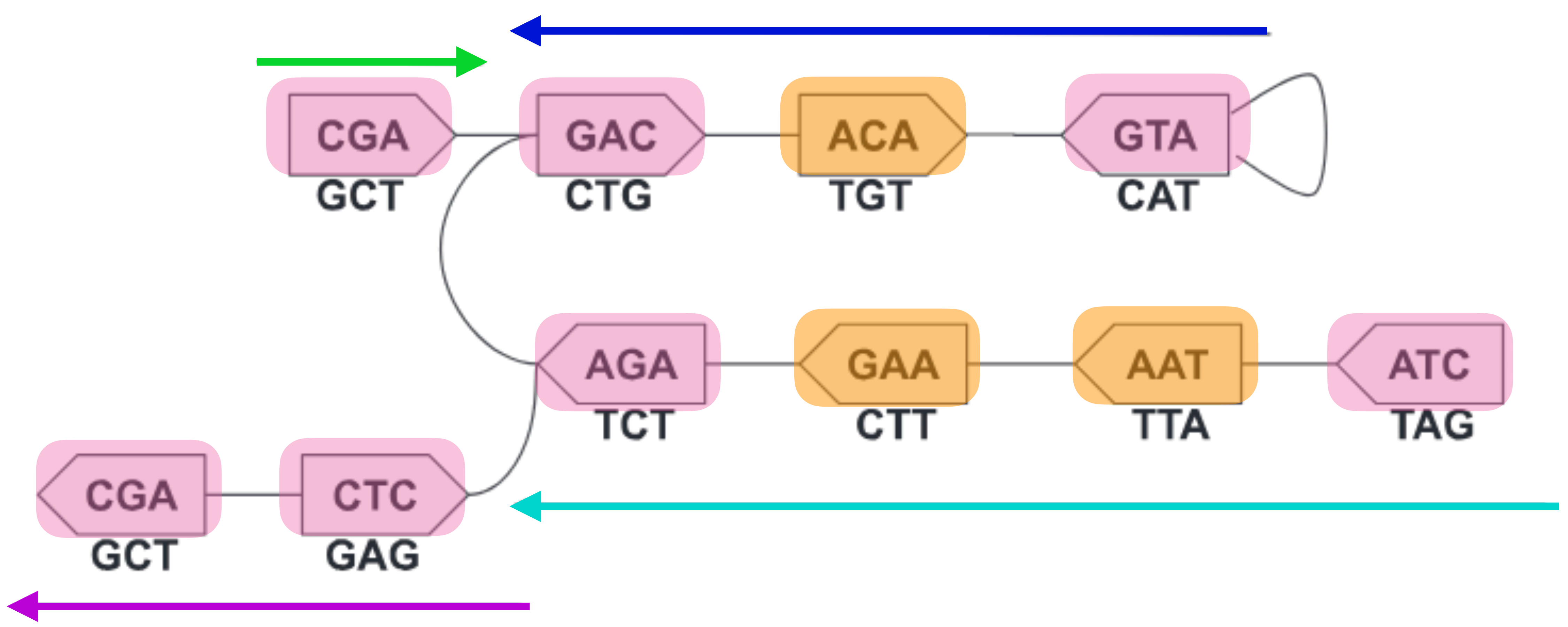
Constructing the DBG efficiently

How to do it

Idea: Implicit *walk* over G(S, k)

There is a *bijection* between maximal unitigs U, and the set of sub-paths bounded by flanking vertices P. [Minkin et al. 2017]

The graph compaction problem can be *reduced to* the problem of determining the set of flanking vertices, and extracting the intervening sequences between them.



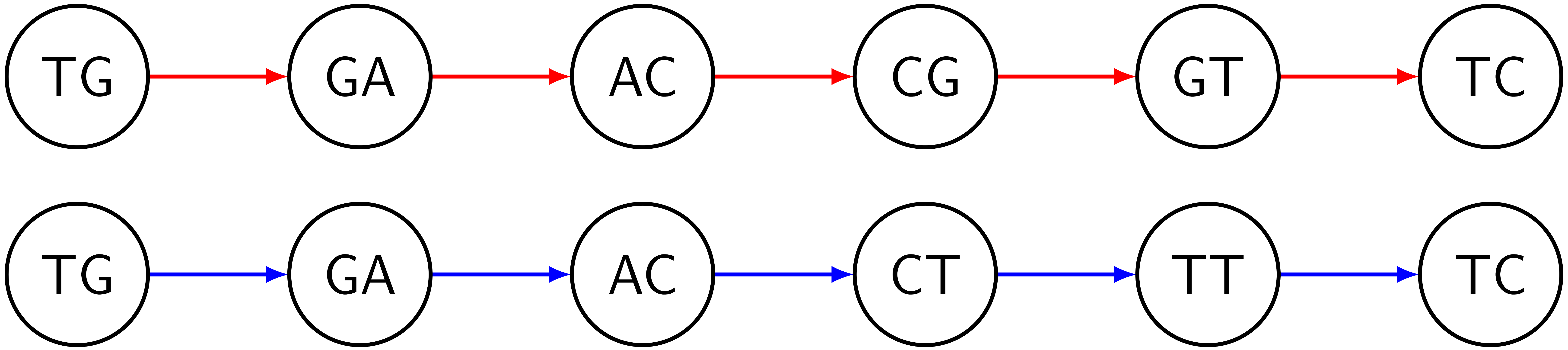
CGA, GAC, ATG, AGA, CTA, AGC, and CTC are flanking

The de Bruijn Graph

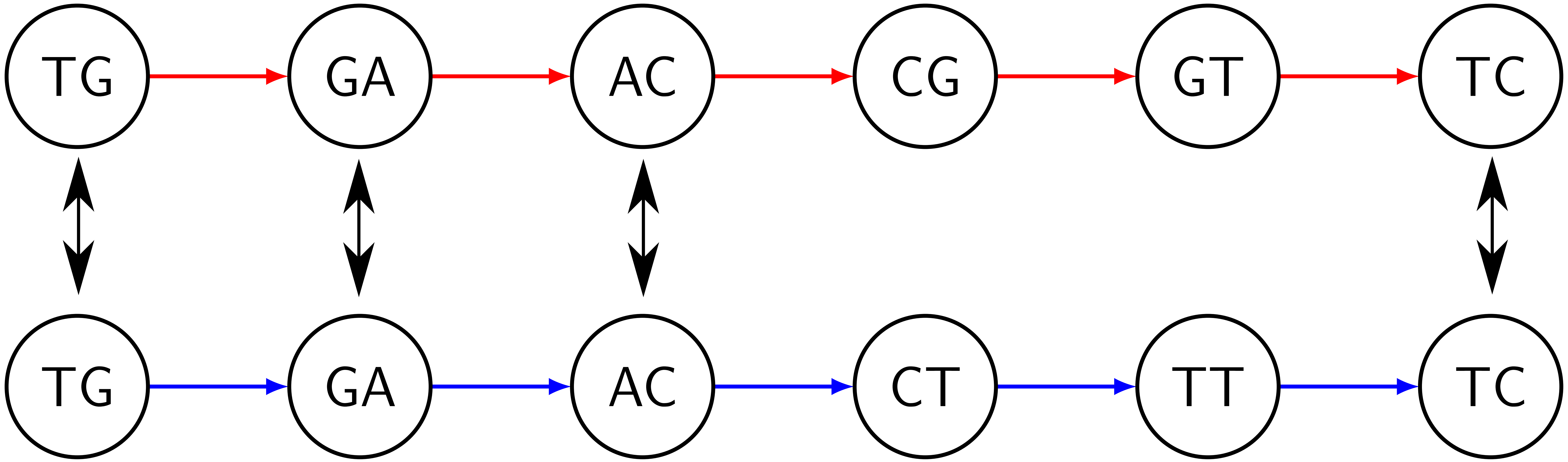
k = 2

TGACGTC

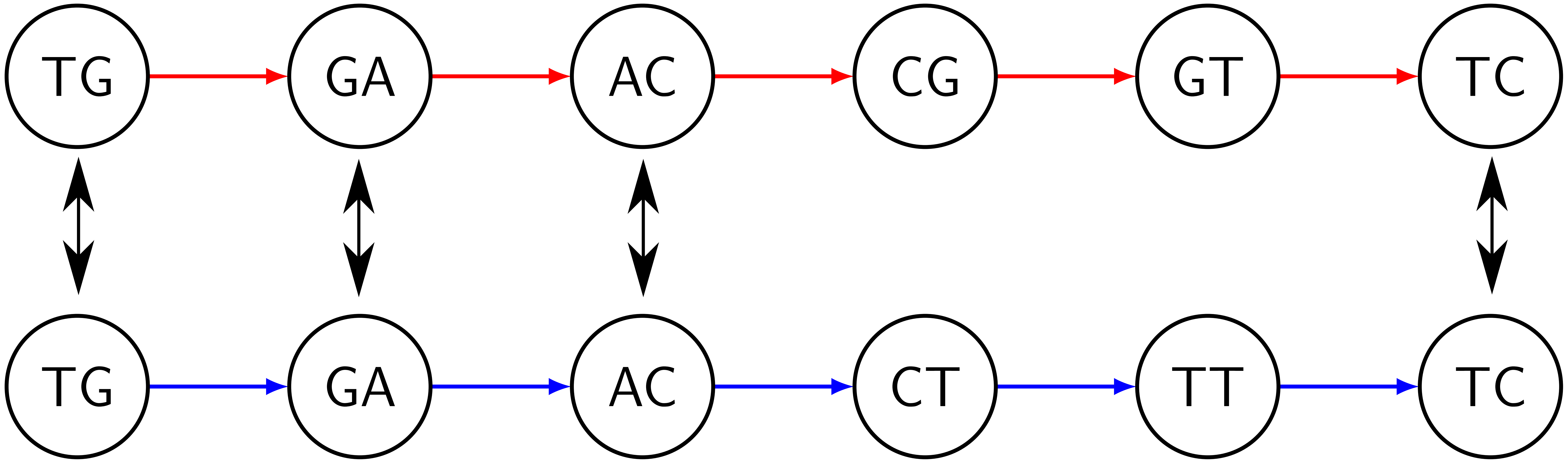
TGACTTC

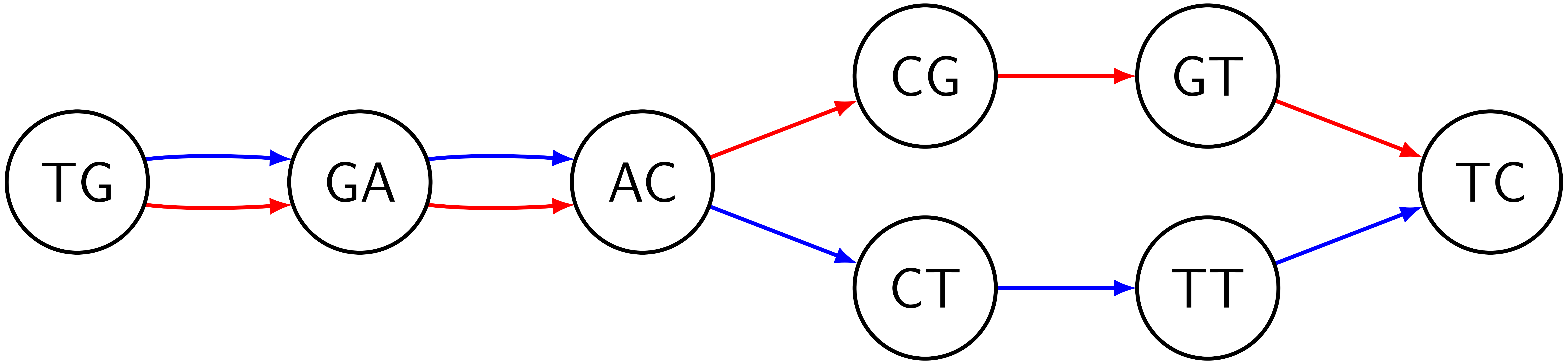


The de Bruijn Graph

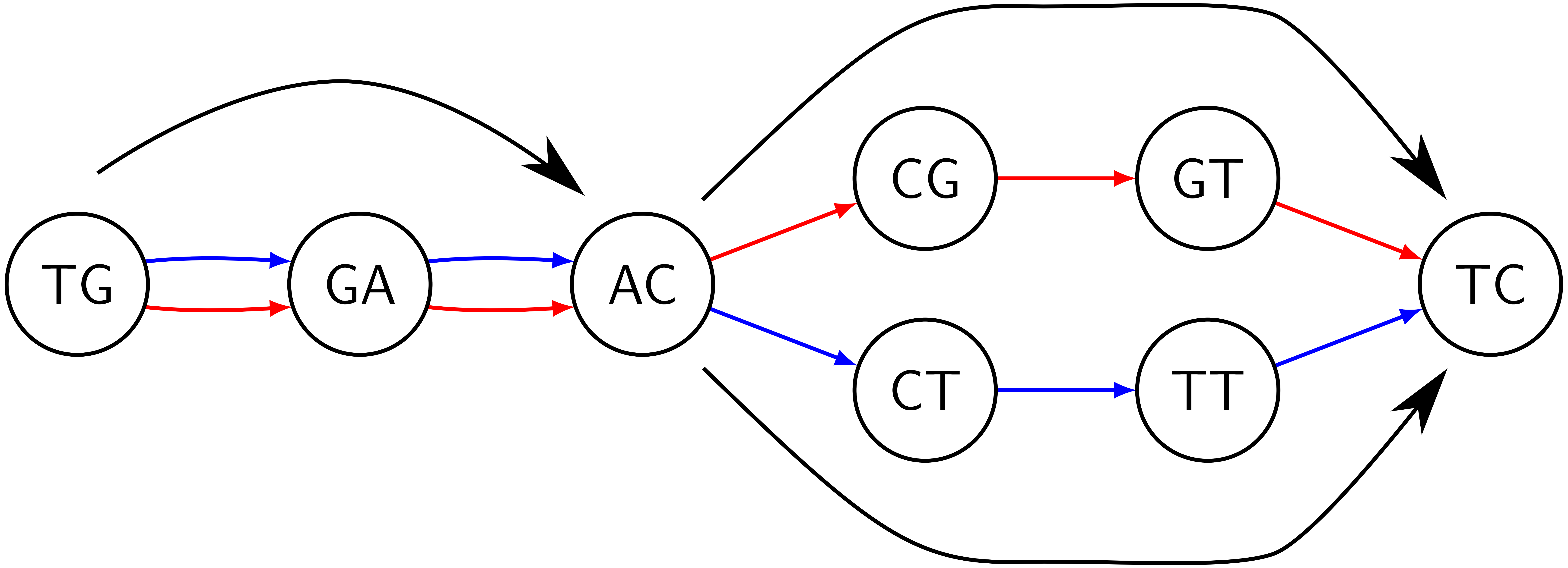


The de Bruijn Graph

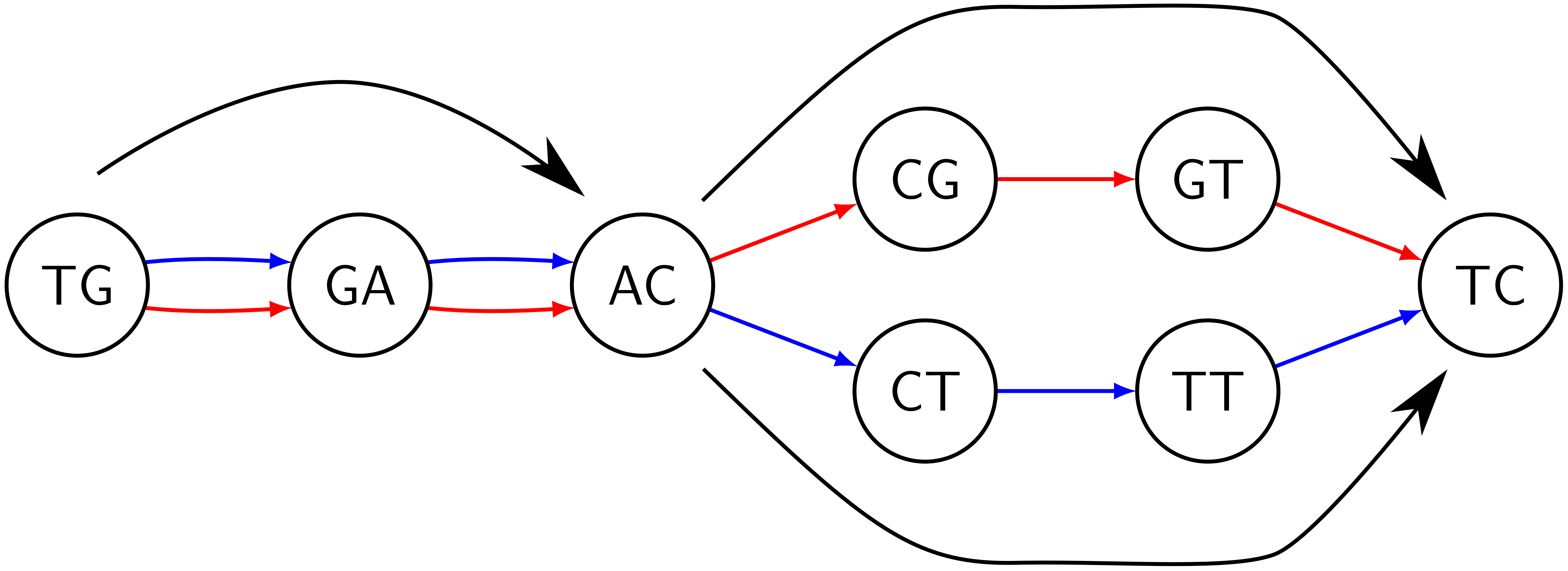




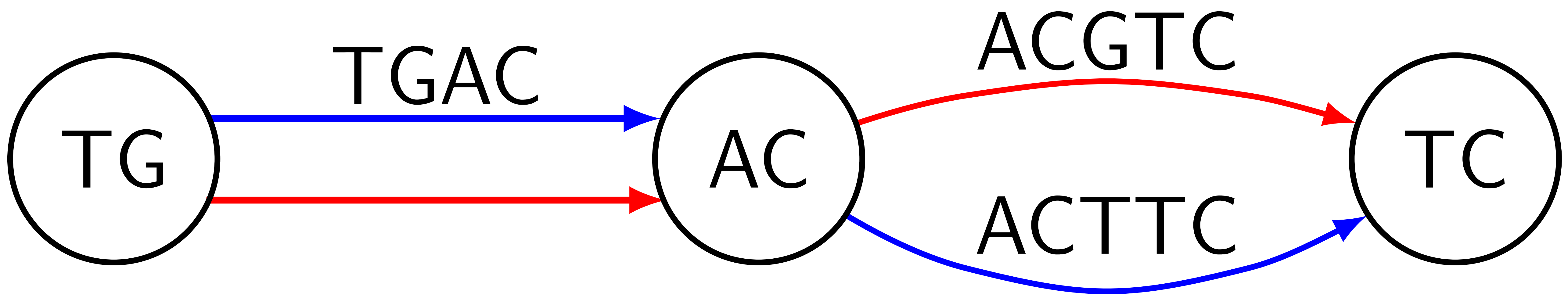
Compaction



Compaction



After compaction:



The Challenge

Construct the compacted graph from many large genomes **bypassing** the ordinary graph traverse.

The Challenge

Construct the compacted graph from many large genomes **bypassing** the ordinary graph traverse.

Earlier work: based on suffix arrays/trees Sibelia & SplitMEM handled >60 E.Coli genomes.

The Challenge

Construct the compacted graph from many large genomes **bypassing** the ordinary graph traverse.

Earlier work: based on suffix arrays/trees Sibelia & SplitMEM handled $>$ 60 E.Coli genomes.

A recent advance: 7 Humans in 15 hours using 100 GB of RAM using a BWT-based algorithm by Baier *et al.*, 2015, Beller *et al.*, 2014.

Junctions

A vertex v is a **junction** if:

- v has ≥ 2 distinct outgoing or incoming edges:

Junctions

A vertex v is a **junction** if:

- v has ≥ 2 distinct outgoing or incoming edges:

- v is the first or the last k-mer of an input string

Junctions

A vertex v is a **junction** if:

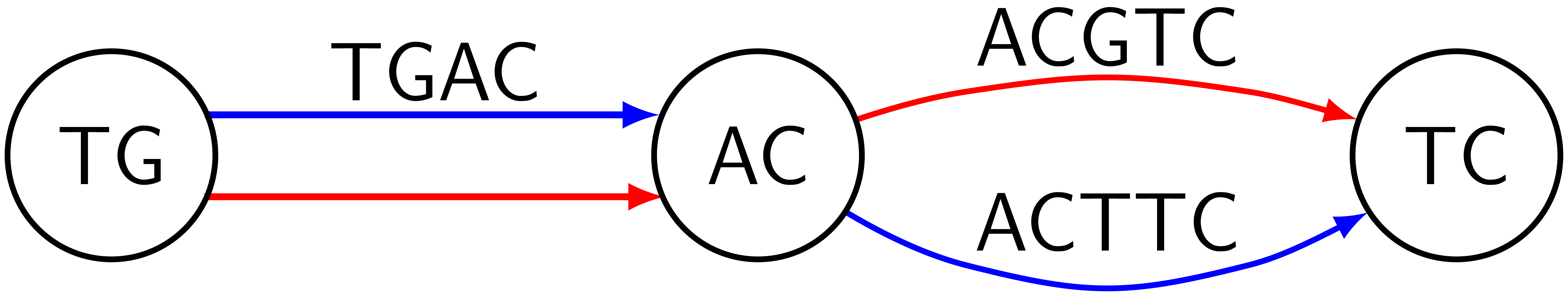
- v has ≥ 2 distinct outgoing or incoming edges:

- v is the first or the last k-mer of an input string

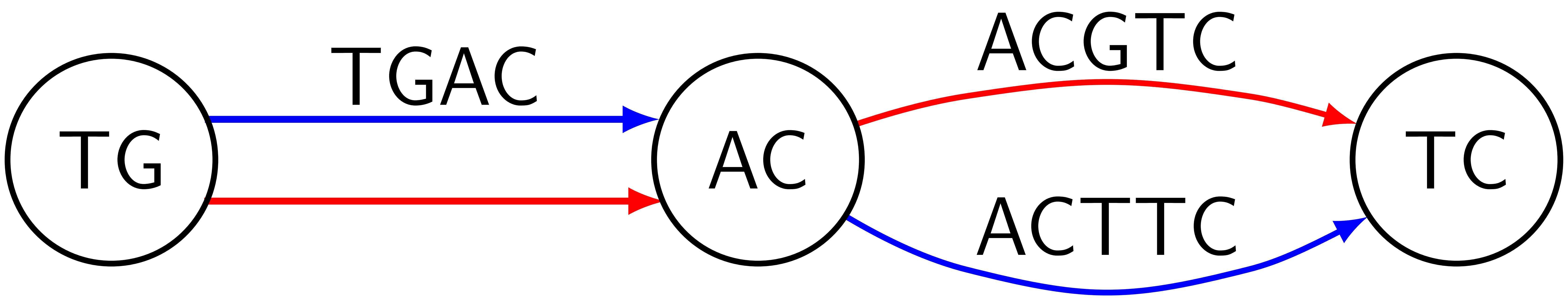
Facts:

- Junctions $=$ vertices of the compacted graph
- Compaction $=$ finding positions of junctions

Observations

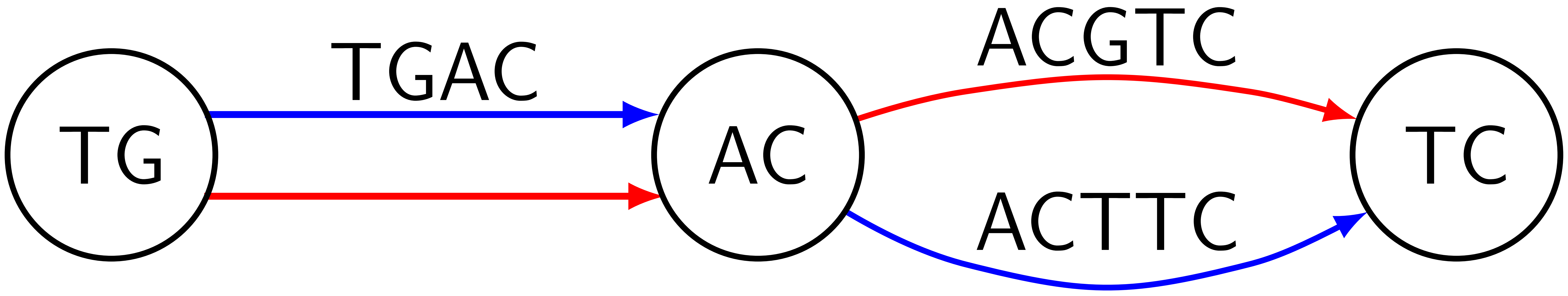


Observations



TG GA AC CG GT TC

Observations



TG GA AC CG GT TC

TG $\rightarrow$ AC $\rightarrow$ TC

The Observation

The observation only works when we have complete genomes.

Once we know junctions, construction of the edges is simple.

We can simply traverse input strings and record junctions in the order they appear.

How to identify junctions?

The Naive Algorithm

A naive way:

- Store all $(k+1)$-mers (edges) in a hash table
- Consider each vertex one by one
- Query all possible edges from the table
- If found > 1 edge, mark vertex as a junction

Simple algorithm in more detail

Algorithm 1. *Filter-Junctions*

Input: strings $S = \{s_1, \ldots, s_n\}$, integer k, and an empty set data structure E. A candidate set of marked junction positions $C \supseteq J(S, k)$ is also given. When the algorithm is run naively, all the positions would be marked.
Output: a reduced candidate set of junction positions.

for $s \in S$ **do**
    for $1 \leq i < |s| - k$ **do**
        if $C[s, i] =$ marked **then** ▷ Insert the two $(k+1)$-mers containing the k-mer at i into E.
            Insert $s[i..i+k]$ into E.
            Insert $s[i-1..i-1+k]$ into E.
for $s \in S$ **do**
    for $1 \leq i < |s| - k$ **do**
        if $C[s, i] =$ marked **and** $s[i..i+k-1]$ is not a sentinel **then**
            $in \leftarrow 0$ ▷ Number of entering edges
            $out \leftarrow 0$ ▷ Number of leaving edges
            for $c \in \{A, C, G, T\}$ **do** ▷ Consider possible edges and count how many of them exist
                if $v \cdot c \in E$ **then** ▷ The symbol $\cdot$ depicts string concatenation
                    $out \leftarrow out + 1$
                if $c \cdot v \in E$ **then**
                    $in \leftarrow in + 1$
            if $in = 1$ **and** $out = 1$ **then** ▷ If the k-mer at i is not a junction.
                $C[s, i] \leftarrow$ Unmarked
return C

The Naive Algorithm

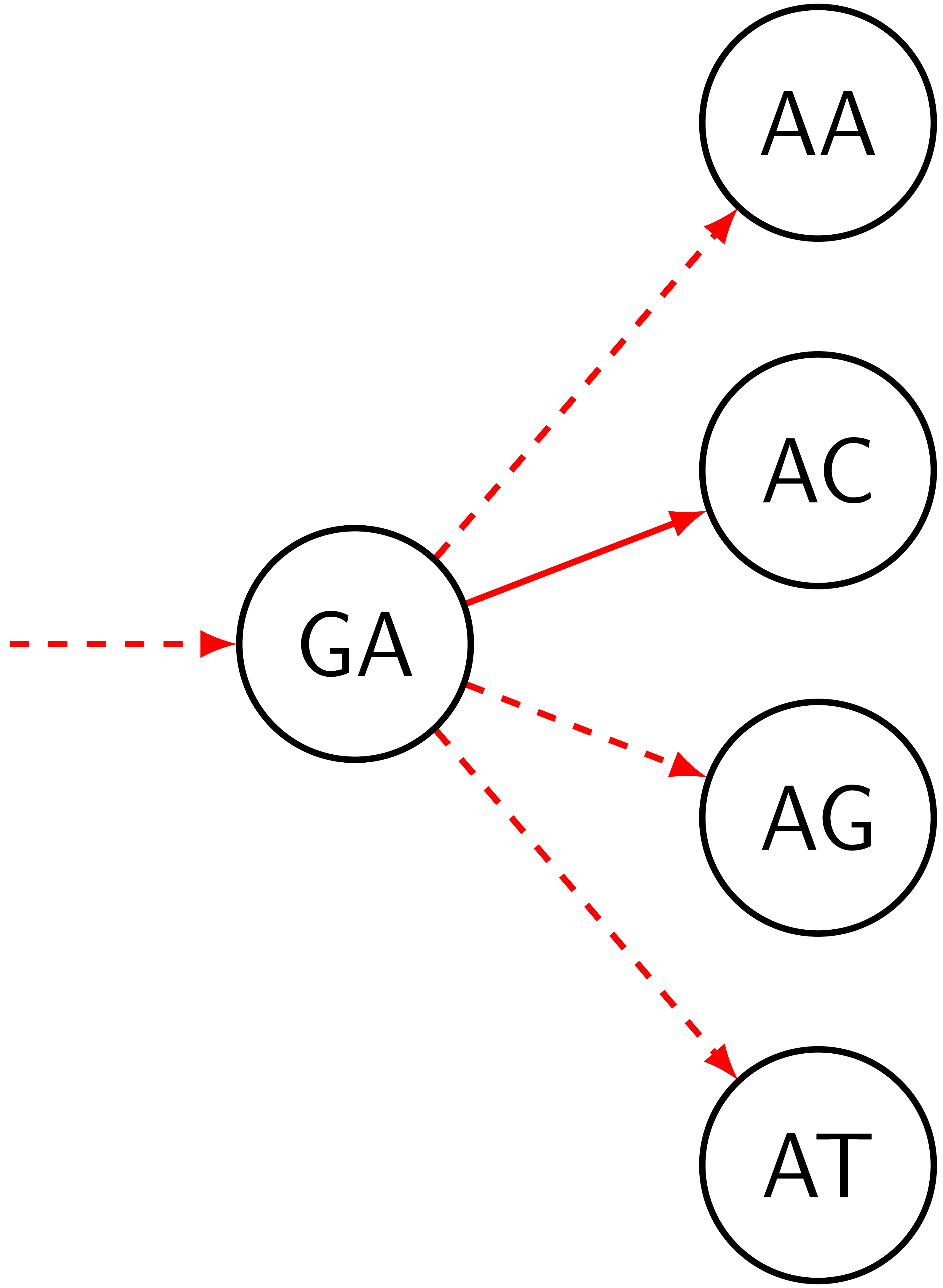
A naive way:

- Store all $(k+1)$-mers (edges) in a hash table
- Consider each vertex one by one
- Query all possible edges from the table
- If found > 1 edge, mark vertex as a junction

Problem: the hash table can be too large.

An Example

Hash table = { GA $\to$ AC }



What is the Bloom filter

A probabilistic data structure representing a set

Properties:

- Occupies fixed space
- May generate false positives on queries
- False positive rate is low

What is the Bloom filter

A probabilistic data structure representing a set

Properties:

- Occupies fixed space
- May generate false positives on queries
- False positive rate is low

Example: Bloom Filter $= \{$ GA $\rightarrow$ AC $\}$

Is GA $\rightarrow$ AC in the set? Yes.

What is the Bloom filter

A probabilistic data structure representing a set

Properties:

- Occupies fixed space
- May generate false positives on queries
- False positive rate is low

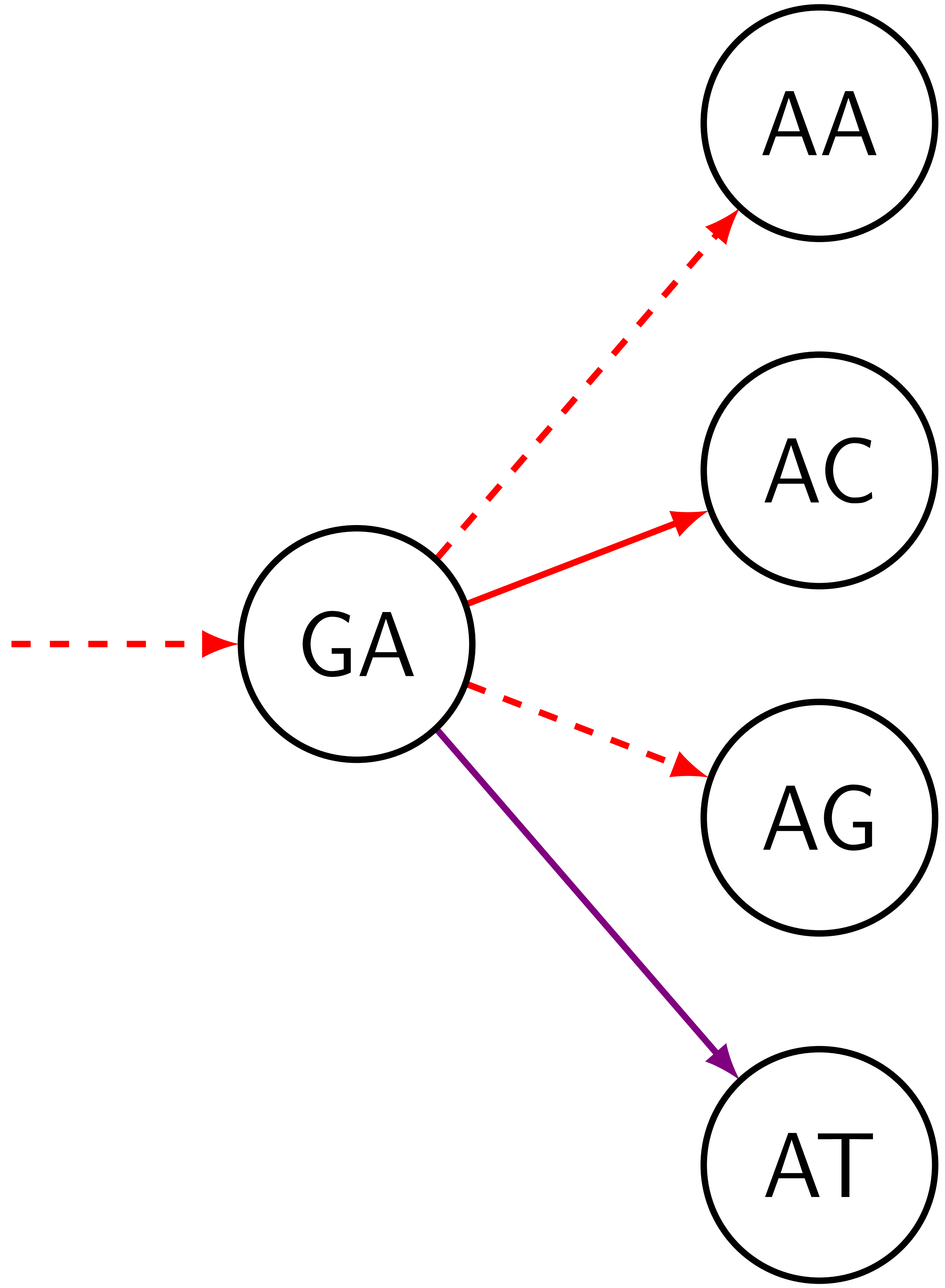
Example: Bloom Filter = { GA $\rightarrow$ AC }

Is GA $\rightarrow$ AC in the set? Yes.

Is GA $\rightarrow$ AT in the set? **Maybe** no.

An Example

Bloom Filter = { GA $\rightarrow$ AC, GA $\rightarrow$ AT }



The purple edge is a false positive.

The Two Pass Algorithm

How to eliminate false positives?

The Two Pass Algorithm

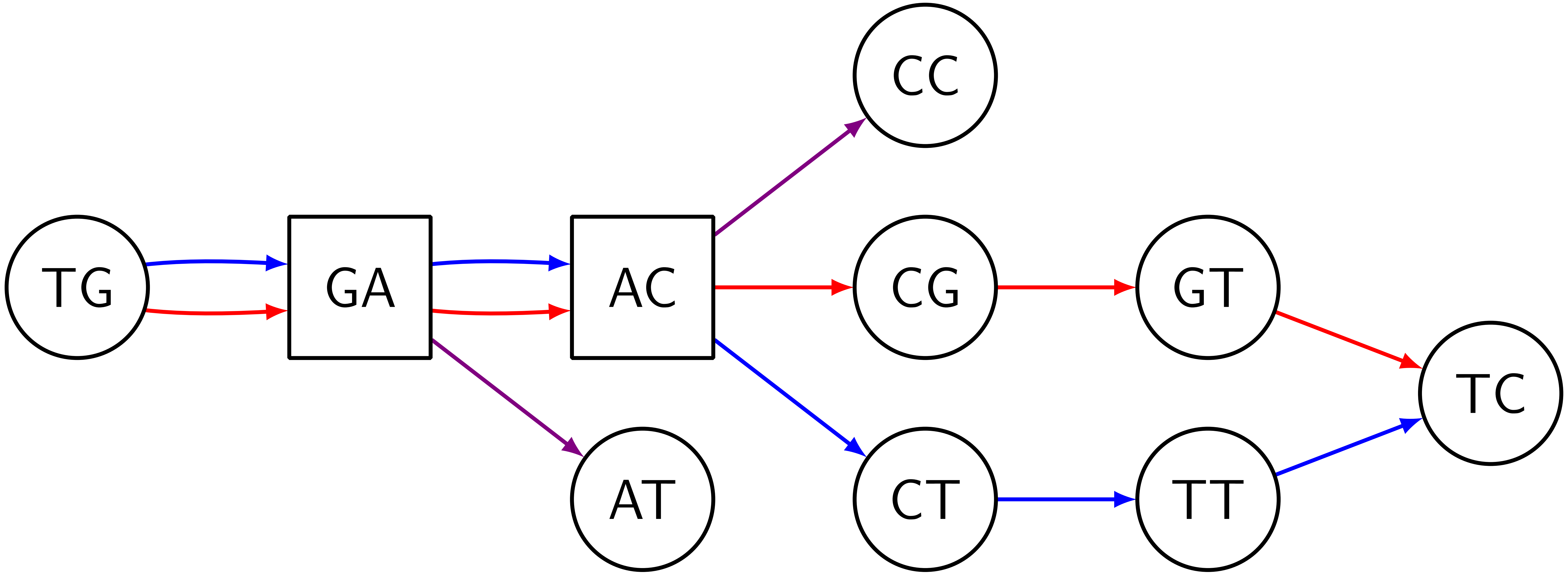
How to eliminate false positives?

Two-pass algorithm:

1. Use the Bloom filter to identify **junction candidates**
2. Use the hash table, but store **only edges that touch candidates**

An Example: the First Step

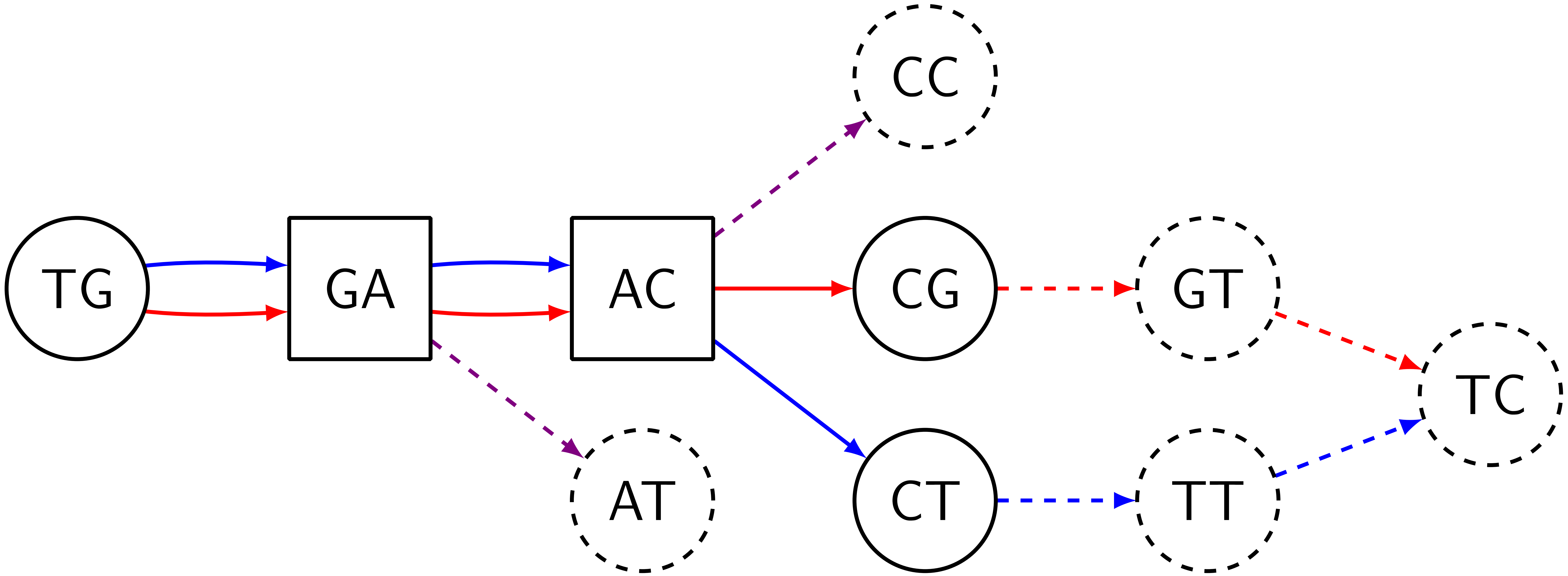
Here edges stored in the Bloom filter, purple ones are false positives:



Junction candidates: GA & AC

An Example: the Second Step

Edges stored in the hash table. We kept only edges touching junction candidates:



Junction: AC

The TwoPass Algorithm

Algorithm 2. Filter-Junctions-Two-Pass

Input: strings $S = \{s_1, \ldots, s_n\}$, integer k, a candidate set of junction positions C_{in}, integer b

Output: a candidate set of junction positions C_{out}

1: $F \leftarrow$ an empty Bloom filter of size b
2: $C_{\text{temp}} \leftarrow \mathit{Filter} - \mathit{Junctions}(S, k, F, C_{\text{in}})$ ▷ The first pass
3: $H \leftarrow$ an empty hash table
4: $C_{\text{out}} \leftarrow \mathit{Filter} - \mathit{Junctions}(S, k, H, C_{\text{temp}})$ ▷ The second pass
5: **return** C_{out}

The TwoPaCo algorithm

Algorithm 3. TwoPaCo

Input: strings $S = \{s_1, \ldots, s_n\}$, integer k, integer ℓ, integer b
Output: the compacted de Bruijn graph $G_c(S, k)$

Initialize counters $c_0, \ldots, c_{q-1}$ to zeroes
$F \leftarrow$ an empty Bloom filter of size b
for $s \in S$ **do**
  for $1 \leq i \leq |s| - k + 1$ **do**
    $h \leftarrow s[i..i + k - 1]$
    if h not in F **then**
      Insert h into F
      $c_{f(h)} \leftarrow c_{f(h)} + 1$
$T \leftarrow \sum_{0 \leq t < q} c_t / \ell$ $\triangleright$ Mean number of k-mers per partition
$p_0 \leftarrow 0, p_\ell \leftarrow q$
for $1 \leq i < \ell$ **do**
  $p_i \leftarrow$ biggest integer larger than p_{i-1} such that $(\sum_{p_{i-1} \leq j < p_i} c_j) \leq T$, or $\min\{\ell, p_{i-1} + 1\}$ if it does not exist.
$C_{\text{init}} \leftarrow$ Boolean array with every position unmarked
for $1 \leq i \leq \ell$ **do**
  $C_i \leftarrow$ mark every position of C_{init} that starts a k-mer h with hash value $p_{i-1} \leq f(h) < p_i$
  $C'_i \leftarrow \text{Filter} - \text{Junctions} - \text{Two} - \text{Pass}(S, k, b, C_i)$
$C_{\text{final}} = \cup C'_i$
return Graph implied by C_{final}, as described in Section 3.

Results

Datasets:

- 7 humans: 5 versions of the reference +
  2 haplotypes of NA12878 from 1000 Genomes
- 93 simulated humans (FIGG)
- 8 primates available in UCSC genome browser

Results

Format: minutes (GB)

Table 2. Benchmarking comparisons

	DSK+BCALM	Minia	Sibelia	SplitMem	bwt-based from Baier *et al.* (2015)		TwoPaCo	
				Single strand	Single strand	Both strands	1 thread	15 threads
62 *E.coli* ($k = 25$)	6 (1.57)	151 (0.9)	10 (12.2)	70 (178.0)	8 (0.85)	12 (1.7)	4 (0.16)	2 (0.39)
62 *E.coli* ($k = 100$)	13 (2.50)	114 (1.9)	8 (7.6)	67 (178.0)	8 (0.50)	12 (1.0)	4 (0.19)	2 (0.39)
7 humans ($k = 25$)	444 (22.44)	968 (48.09)	–	–	867 (100.30)	1605 (209.88)	436 (4.40)	63 (4.84)
7 humans ($k = 100$)	1347 (221.65)	1857 (222.0)	–	–	807 (46.02)	1080 (92.26)	317 (8.42)	57 (8.75)
8 primates ($k = 25$)	2088 (85.62)	–	–	–	–	–	914 (34.36)	111 (34.36)
8 primates ($k = 100$)	–	–	–	–	–	–	756 (56.06)	101 (61.68)
$(43 + 7)$ humans ($k = 25$)	–	–	–	–	–	–		705 (69.77)
$(43 + 7)$ humans ($k = 100$)	–	–	–	–	–	–		927 (70.21)
$(93 + 7)$ humans ($k = 25$)	–	–	–	–	–	–		1383 (77.42)

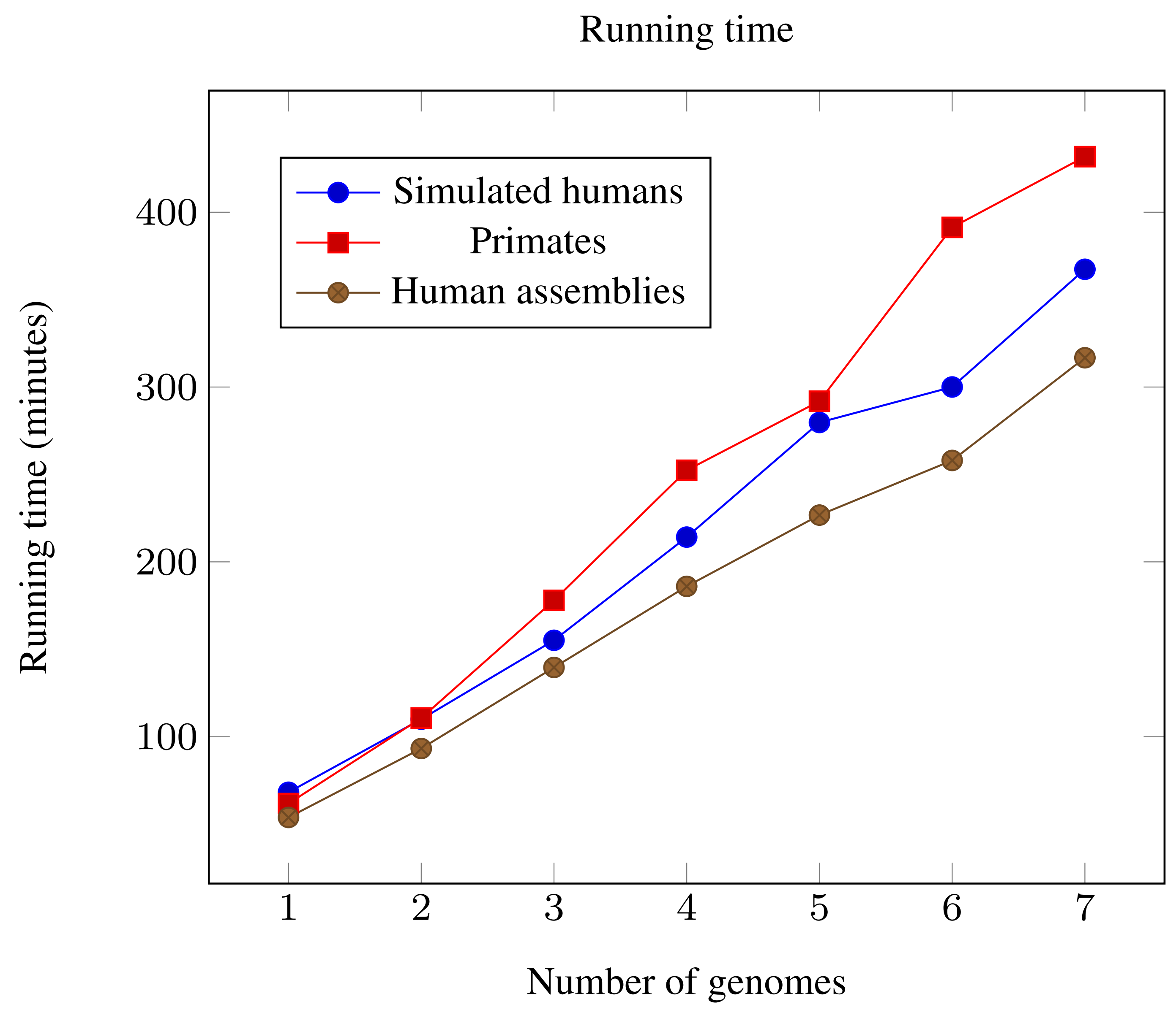
Note: Each cell shows the running time in minutes and the memory usage in parenthesis in gigabytes. TwoPaCo was run using just one round, with a Bloom filter size $b = 0.13$ GB for *E.coli*, 4.3 GB for 7 humans with $k = 25$, $b = 8.6$ GB with $k = 100$, $b = 34$ GB for primates, and $b = 69$ GB for $(43 + 7)$ and larger human dataset. A dash in the SplitMem and bwt-based columns indicates that they ran out of memory, a dash in the Sibelia column indicates that it could not be run on such large inputs, a dash in the minia column indicates that it did not finish in 48 h, a dash in the BCALM column indicates that it ran out of disk space (4 TB). A double dash indicates that the software had a segmentation fault. An empty slot indicates that the experiment was not done.

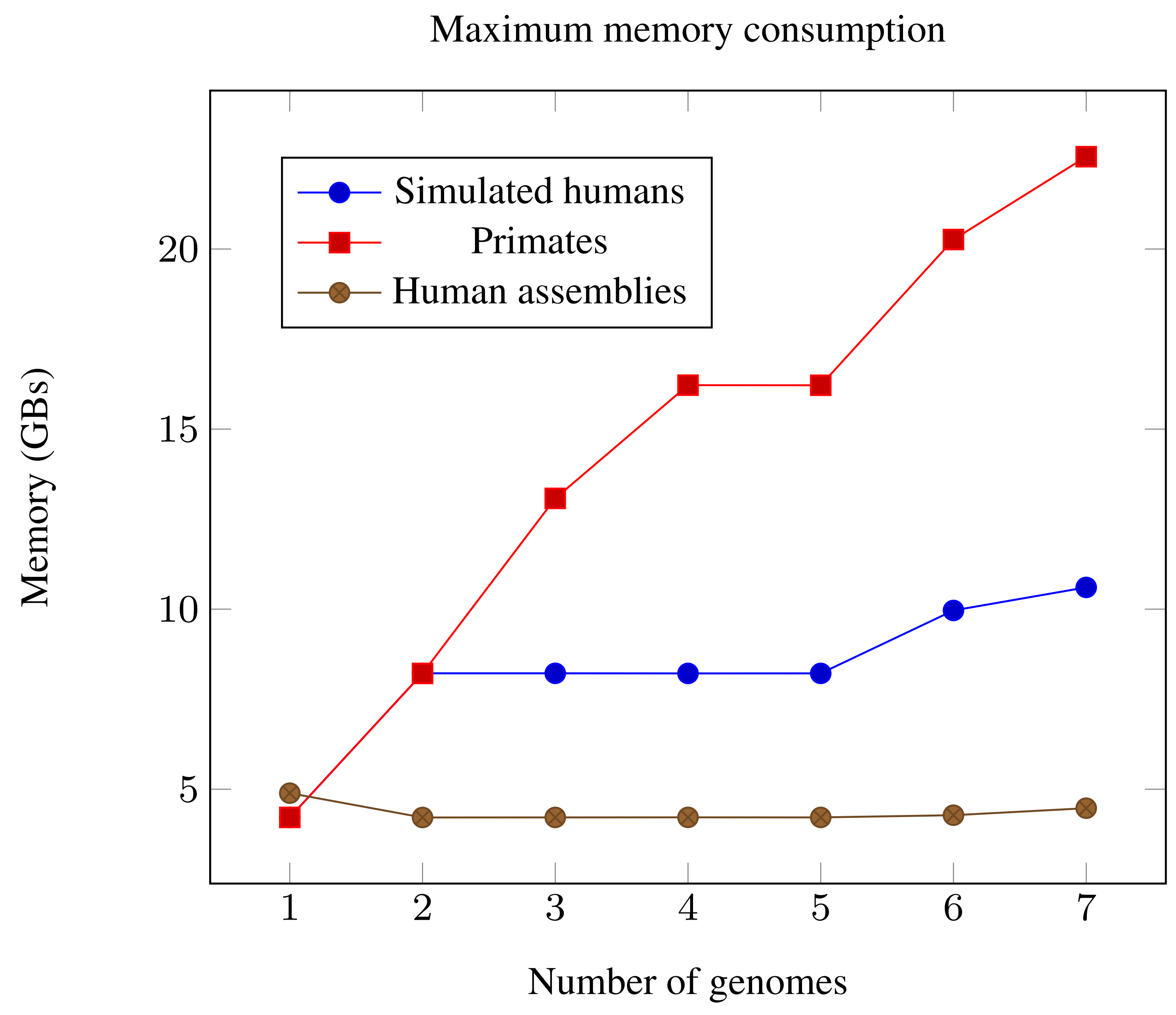
Conclusion & Future Work

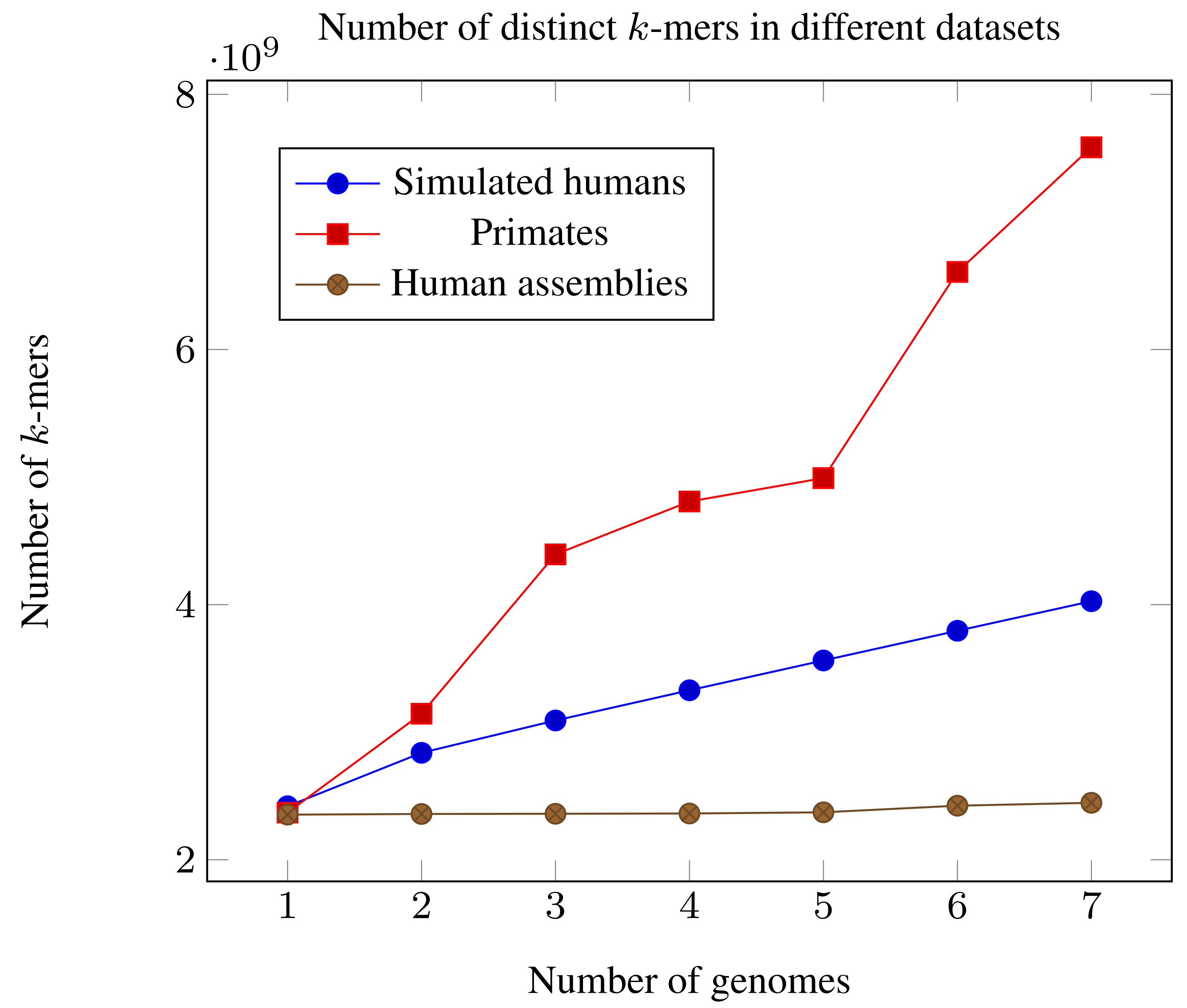
Can potentially facilitate:

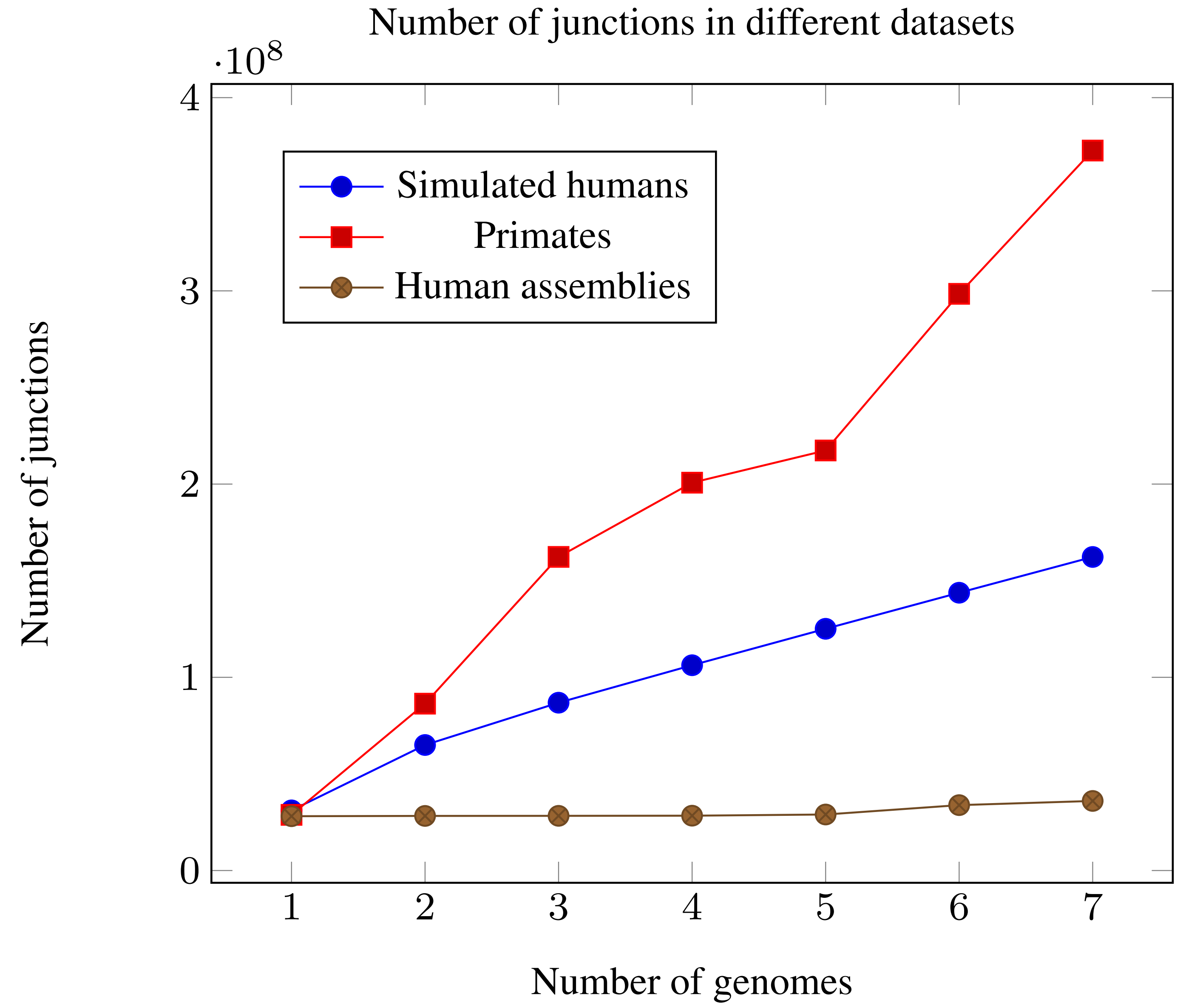
- Visualization
- Synteny mining (Sibelia)
- Structural variations analysis
- ...

Input Size vs. Performance

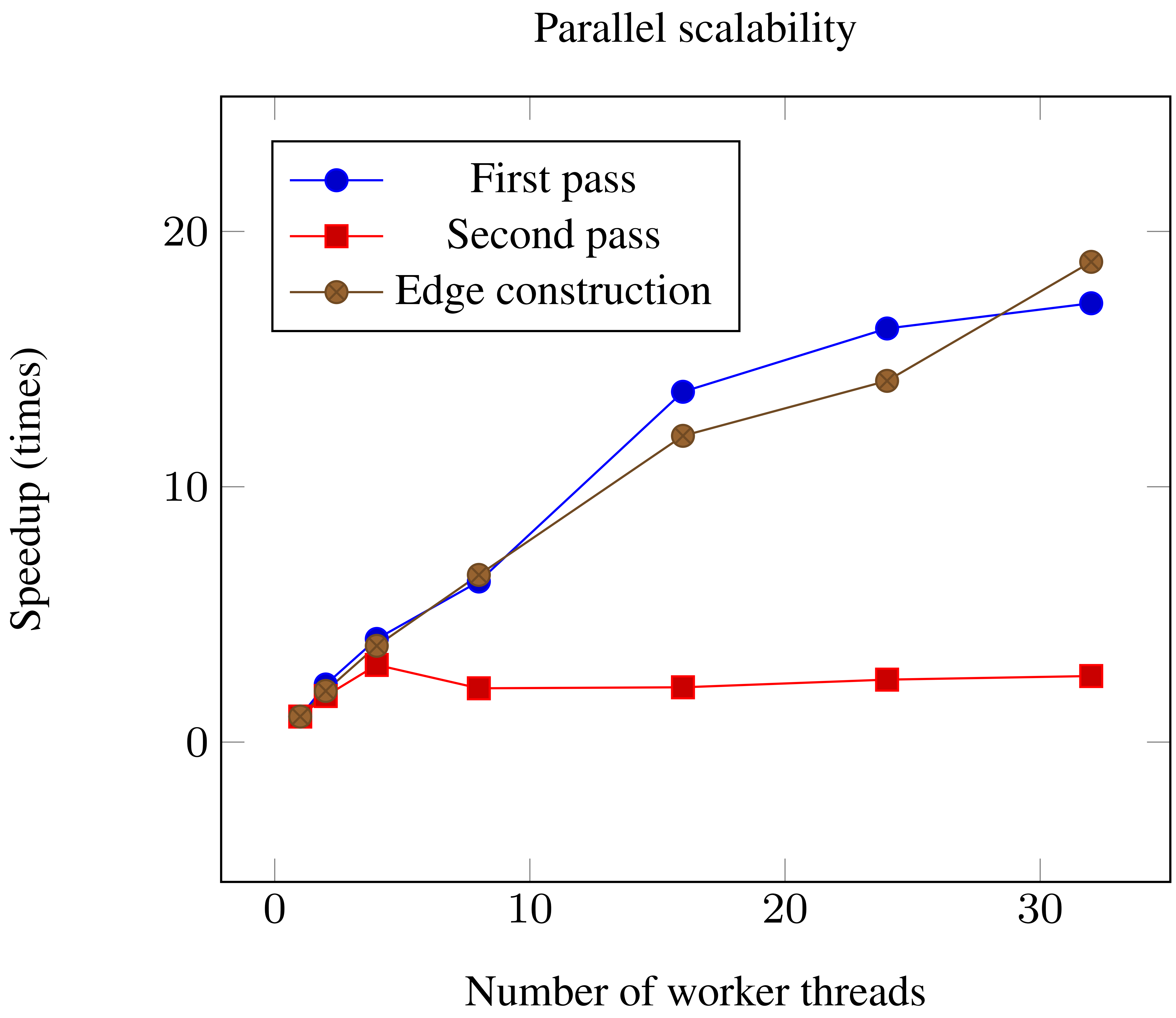








Parallel Scalability



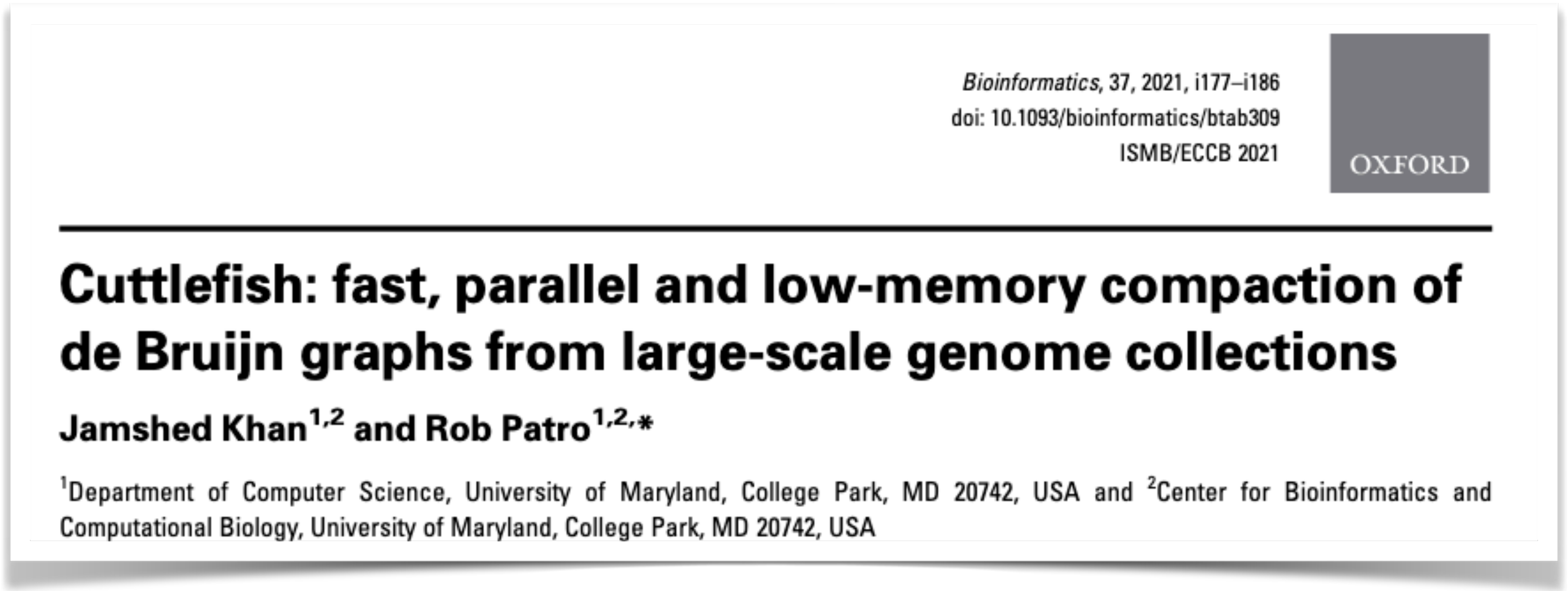
Splitting

Table 1: The minimal number of rounds it takes to compress the graph without exceeding a given memory threshold.

Memory threshold	Used memory	Bloom filter size	Running time	Rounds
10	8.62	8.59	259	1
8	6.73	4.29	434	3
6	5.98	4.29	539	4
4	3.51	2.14	665	6

Constructing the DBG efficiently

How to do it


Bioinformatics, 37, 2021, i177–i186
doi: 10.1093/bioinformatics/btab309
ISMB/ECCB 2021
OXFORD

Cuttlefish: fast, parallel and low-memory compaction of de Bruijn graphs from large-scale genome collections

Jamshed Khan[1,2] and Rob Patro[1,2,*]

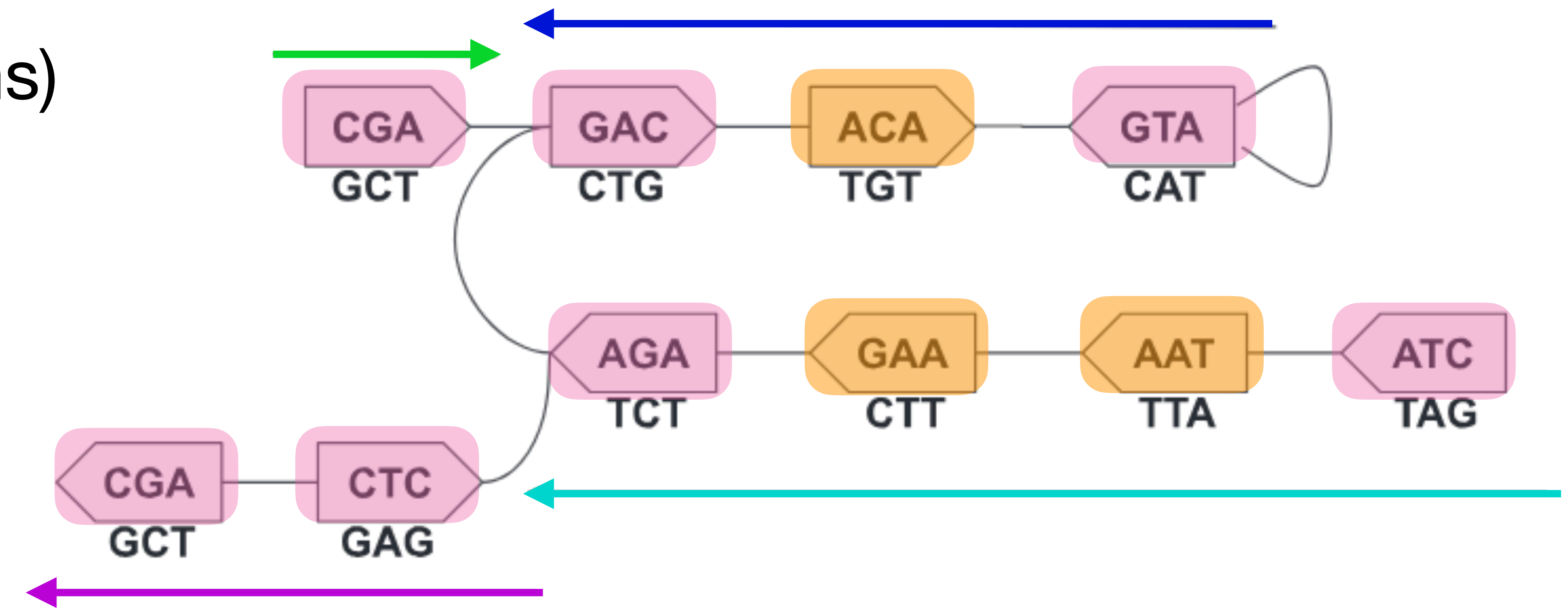
[1]Department of Computer Science, University of Maryland, College Park, MD 20742, USA and [2]Center for Bioinformatics and Computational Biology, University of Maryland, College Park, MD 20742, USA

Idea: Implicit *walk* over G(S, k)

For an *edge-centric* DBG, a complete walk traversal w(s) over G(s, k) can be obtained via a scan over s, *without having G(s, k) itself present in memory*

and the maximal unitigs (i.e. maximal non-branching paths) of G(s, k) are contained as subpaths in this walk w(s)

If each $v \in G(S, k)$ can be characterized as "flanking" or "internal" (w.r.t each maximal unitig), the unitig sequences themselves can be extracted by identifying subpaths having flanking vertices at both ends.



CGA, GAC, ATG, AGA, CTA, AGC, and CTC are flanking

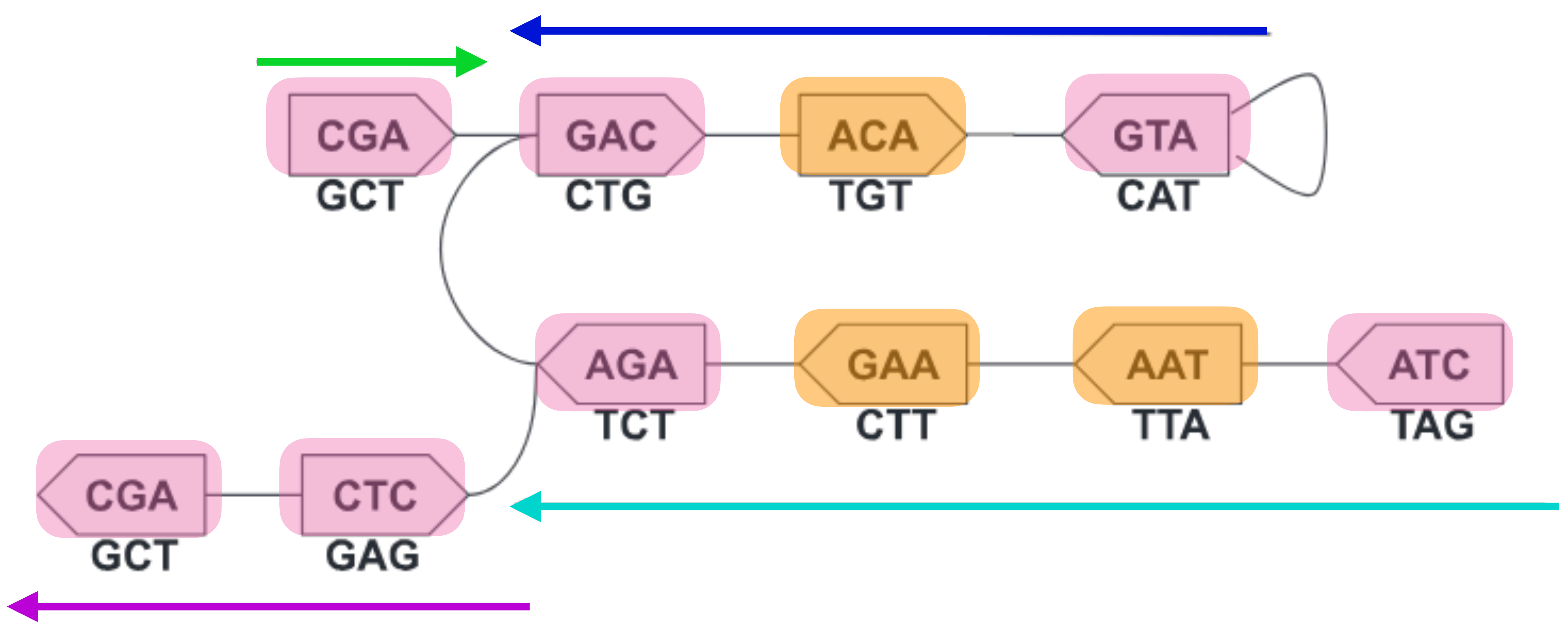
Constructing the DBG efficiently

How to do it

Idea: Implicit *walk* over G(S, k)

There is a *bijection* between maximal unitigs U, and the set of sub-paths bounded by flanking vertices P. [Minkin et al. 2017]

The graph compaction problem can be *reduced to* the problem of determining the set of flanking vertices, and extracting the intervening sequences between them.



CGA, GAC, ATG, AGA, CTA, AGC, and CTC are flanking

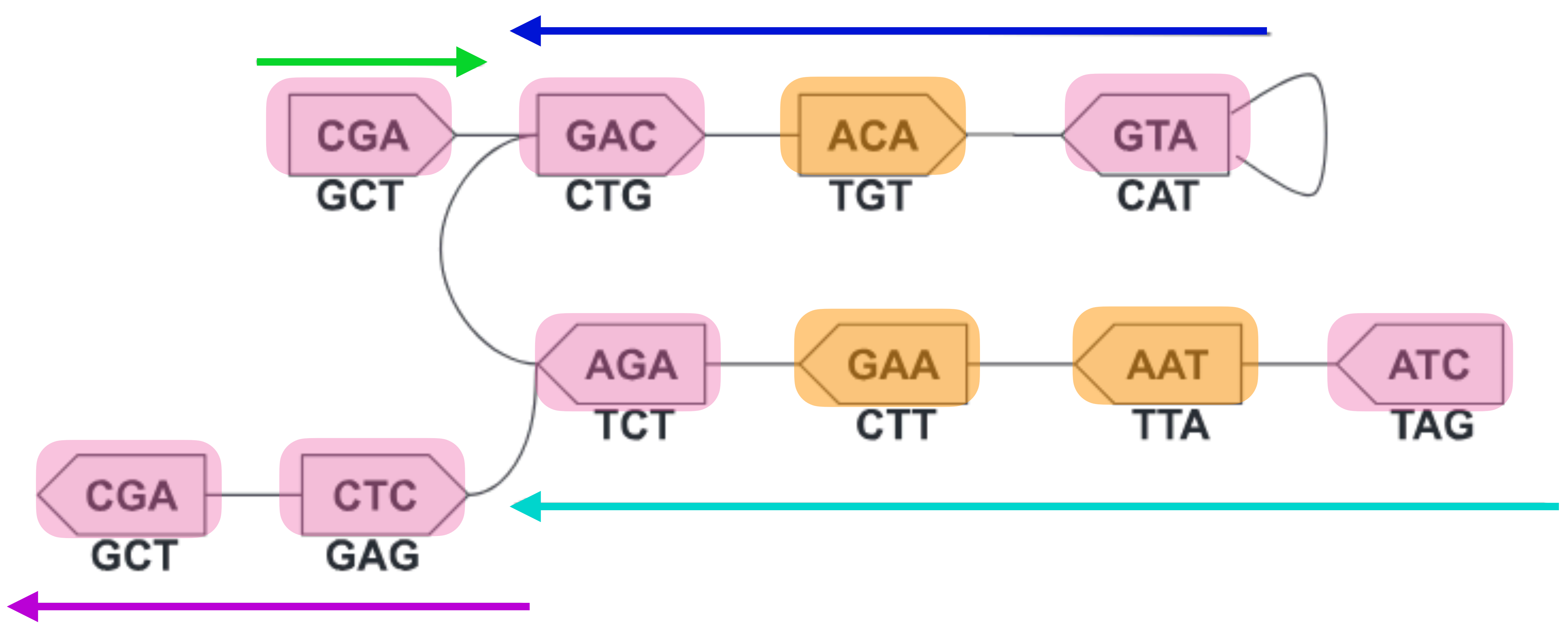
Constructing the DBG efficiently

How to do it

A vertex v in G(S, k) is a *flanking vertex* if it has a side s_v with:

1. 0 or > 1 incident edges (if 0 we refer to it as "sentinel" or "terminal")

2. Or, exactly one edge (v, s_v, u, s_u), and s_u has > 1 incident edge (i.e. its neighbor is branching).



CGA, GAC, ATG, AGA, CTA, AGC, and CTC are flanking

Constructing the DBG efficiently

How to do it

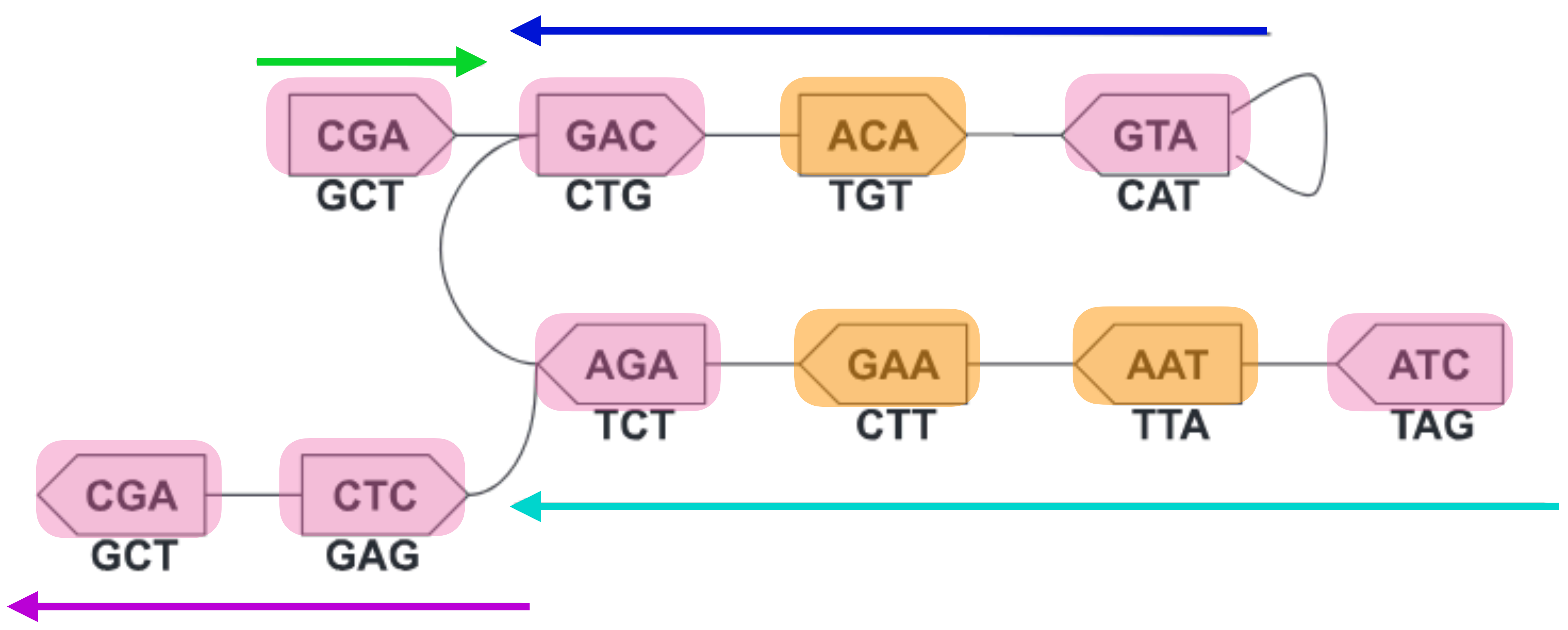
In G(S, k), a side of a vertex can be considered to be in one of five different configurations

- one for each unique singleton edge ($\left|\Sigma\right| = \left|\{A, C, G, T\}\right| = 4$ possible)
- one for when it has $\neq 1$distinct edges

Any other adjacency information is irrelevant for determining flanking status.

Thus, a 2-sided vertex can be in one of (5 x 5) = 25 configurations (or states).

Additionally one extra or “unvisited” state.



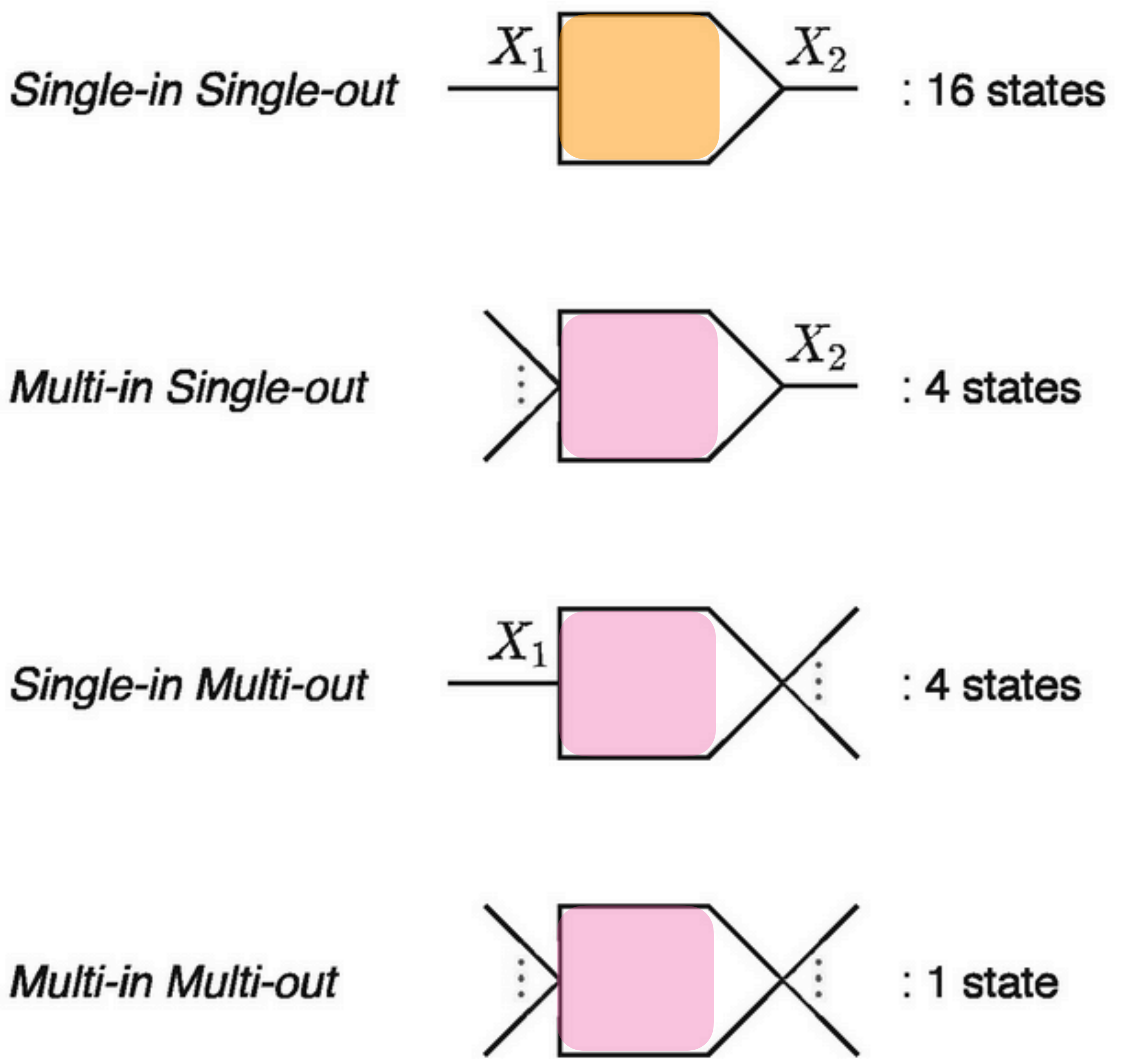
CGA, GAC, ATG, AGA, CTA, AGC, and CTC are flanking

Constructing the DBG efficiently

How to do it

In Cuttlefish, each vertex is treated as a deterministic finite-state automaton (DFA) : $(Q, \Sigma', \delta, q_0, Q')$

- Σ' is the input alphabet for transitions
- Q is the set of possible 25 states that can be partitioned into four disjoint classes
- q_0 is the special initial (unvisited) state
- Q' is the set of final "accepting" states

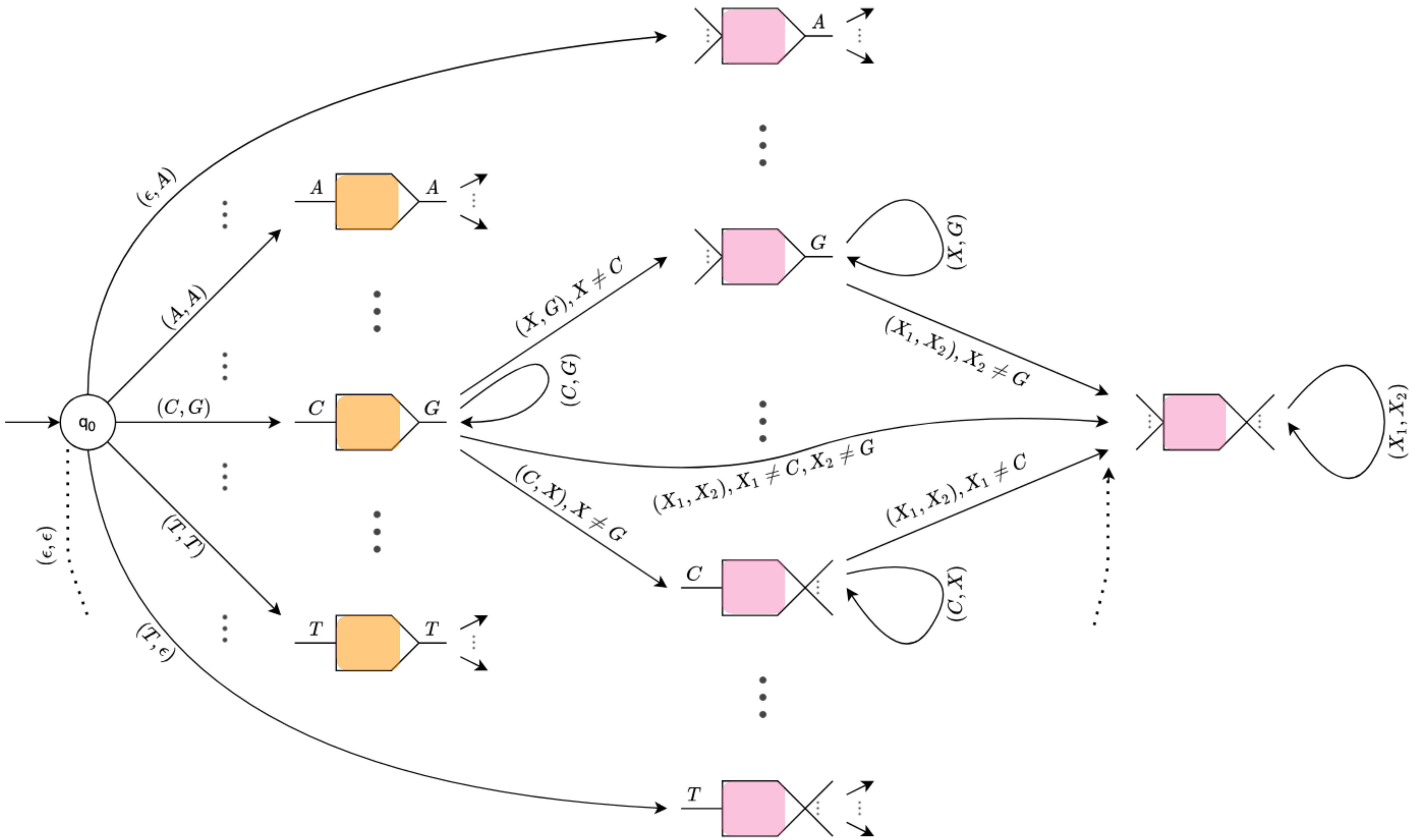


Constructing the DBG efficiently

How to do it

In Cuttlefish, each vertex is treated as a deterministic finite-state automaton (DFA) : $(Q, \Sigma', \delta, q_0, Q')$

- δ is the transition function
- Eliding specifics for time, simply encodes for each vertex state, given new observation of adjacent edge, the new state to assume.



Constructing the DBG efficiently

How to do it

In Cuttlefish, we maintain:

1. A minimal perfect hash function* (as the set of keys = k-mers is static) taking ~3.7 bits / kmer
2. An *atomic* bit-packed array for the hash buckets taking $\lceil \log_2(26) \rceil = 5$ bits/k-mer

```
Cuttlefish(S):
  K = Extract-Unique-k-mers(S)
  h = Construct-Minimal-Perfect-Hash(K)
  // simply done (in parallel) by scanning S and transitioning each
  // vertex according to δ (can be done in parallel across many threads!)
  B = Compute-States(S, h, |K|)
  for each s in S:
    // noting that B now contains the final “flanking” status of each vertex
    Extract-Maximal-Unitigs(s, h, B)
```

The expected running time of Cuttlefish is $O(\lceil \frac{k}{32} \rceil m + nH(k) + mH(k)) = O((m+n)H(k))$

k: k-mer length, m: # edges, n: # vertices, H(x): time to hash a k-mer

*[Limasset et al. 17] (advertisement for CMSC701😁)

Constructing the DBG efficiently

Results

<u>Individual reference genomes:</u>

- Human (~3.2 Gbp)
- Western gorilla (~3 Gbp)
- Sugar pine (~27.6 Gbp)

<u>Collections of references:</u>

- 62 E. coli (~310 Mbp)
- 7 humans (~21 Gbp)
- 7 apes (~18 Gbp)
- 11 conifer plans (~204 Gbp)
- 100 humans (~322 Gbp)

Constructing the DBG efficiently

Results (individual genomes)

Dataset	Thread-count	k	Bifrost Build	Bifrost Output	deGSM Build	deGSM Output	TwoPaCo Build	TwoPaCo Output	Cuttlefish Build	Cuttlefish Output
Human	1	31	04:54:50 (27.23)	15:18	01:54:41 (37.94)	25:06 (9.79)	01:13:19 (4.15)	39:38 (4.50)	32:59 (2.79)	19:23 (2.84)
		61	05:16:51 (50.19)	01:49	02:20:57 (84.16)	21:37 (8.77)	01:10:18 (6.02)	12:25 (4.35)	38:21 (3.06)	15:37 (3.08)
	8	31	01:33:54 (27.23)	03:59	25:20 (37.94)	05:37 (9.80)	12:57 (5.04)	—	05:49 (2.79)	05:13 (2.92)
		61	01:20:28 (50.18)	00:40	47:52 (84.16)	03:55 (8.80)	11:28 (5.46)	—	07:45 (3.06)	03:20 (3.18)
	16	31	01:24:40 (27.24)	03:30	18:19 (37.94)	03:56 (9.80)	06:24 (5.57)	—	03:26 (2.79)	02:57 (2.93)
		61	01:12:33 (50.18)	00:52	46:34 (84.16)	02:35 (8.80)	07:12 (5.55)	—	04:23 (3.06)	01:54 (3.19)
Gorilla	1	31	05:44:10 (28.08)	16:30	01:34:29 (37.94)	24:26 (9.75)	01:00:15 (5.04)	43:25 (4.49)	31:46 (2.74)	17:07 (2.77)
		61	05:31:06 (50.13)	02:05	02:11:33 (84.16)	22:03 (8.94)	01:11:29 (5.83)	17:52 (4.30)	38:15 (3.02)	15:59 (3.03)
	8	31	02:06:52 (28.08)	03:44	28:52 (37.94)	05:43 (9.76)	13:02 (5.82)	—	05:30 (2.74)	04:37 (2.87)
		61	01:24:21 (50.13)	00:54	47:45 (84.16)	03:59 (8.98)	10:03 (6.00)	—	07:58 (3.02)	02:54 (3.12)
	16	31	01:50:26 (28.08)	02:59	20:47 (37.94)	04:07 (9.76)	07:29 (5.52)	—	03:13 (2.74)	03:25 (2.87)
		61	01:10:06 (50.13)	04:04	38:45 (84.16)	02:40 (8.98)	06:24 (6.09)	—	04:29 (3.02)	02:06 (3.14)
Sugar pine	16	31	22:18:24 (229.17)	01:20:51	09:29:24 (145.23)	01:10:55 (119.18)	01:49:01 (61.93)	—	51:30 (14.24)	01:56:52 (14.28)
		61	X (364.25)	—	X (166.54)	—	01:26:39 (64.86)	—	03:14:44 (20.88)	01:26:26 (20.90)

Time in HH:MM:SS; Peak RSS (GB)

Best value in each row in

Constructing the DBG efficiently

Results (genomes collections)

Dataset	Total genome-length (bp)	Distinct k-mers count	Bifrost	deGSM	TwoPaCo	Cuttlefish
62 *E.coli*	310 M	24 M	1 (0.47)	1 (3.34)	1 (0.80)	1 (0.96)
7 Humans	21 G	2.6 B	95 (29.06)	30 (37.94)	62 (6.14)	21 (2.88)
7 Apes	18 G	7.1 B	294 (100.25)	172 (145.23)	59 (28.87)	25 (7.42)
11 Conifers	204 G	82 B	—	—	981 (288.99)	525 (84.12)
100 Humans	322 G	28 B	—	—	1395 (126.25)	523 (28.75)

Time in minutes; Peak RSS (GB)

Best value in each row in

All tests with 16 threads & k = 31

Constructing the DBG (even more) efficiently

METHOD — Open Access

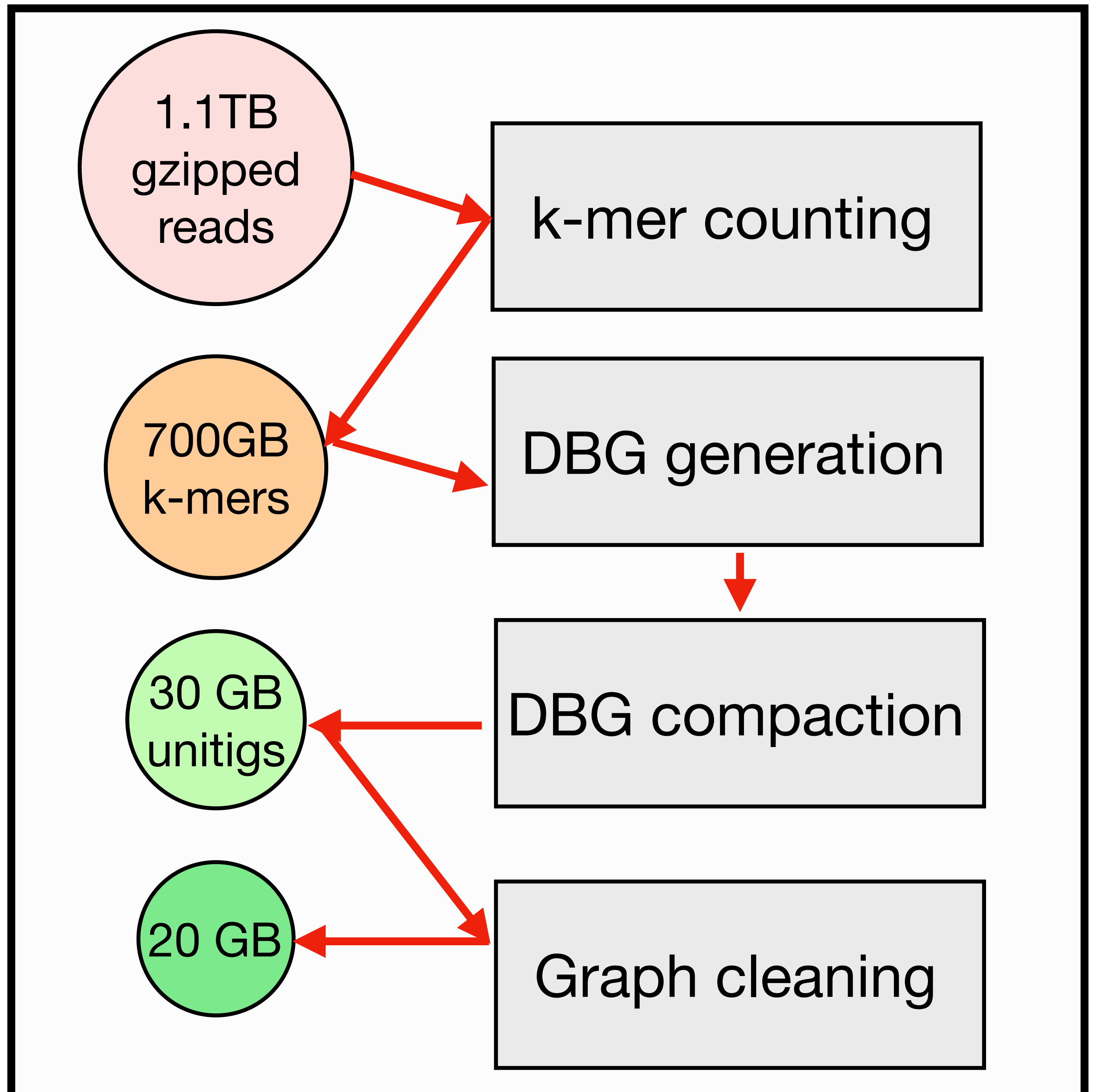
Scalable, ultra-fast, and low-memory construction of compacted de Bruijn graphs with Cuttlefish 2

Jamshed Khan[1,2], Marek Kokot[3*], Sebastian Deorowicz[3] and Rob Patro[1,2*]

<u>Key improvements:</u>

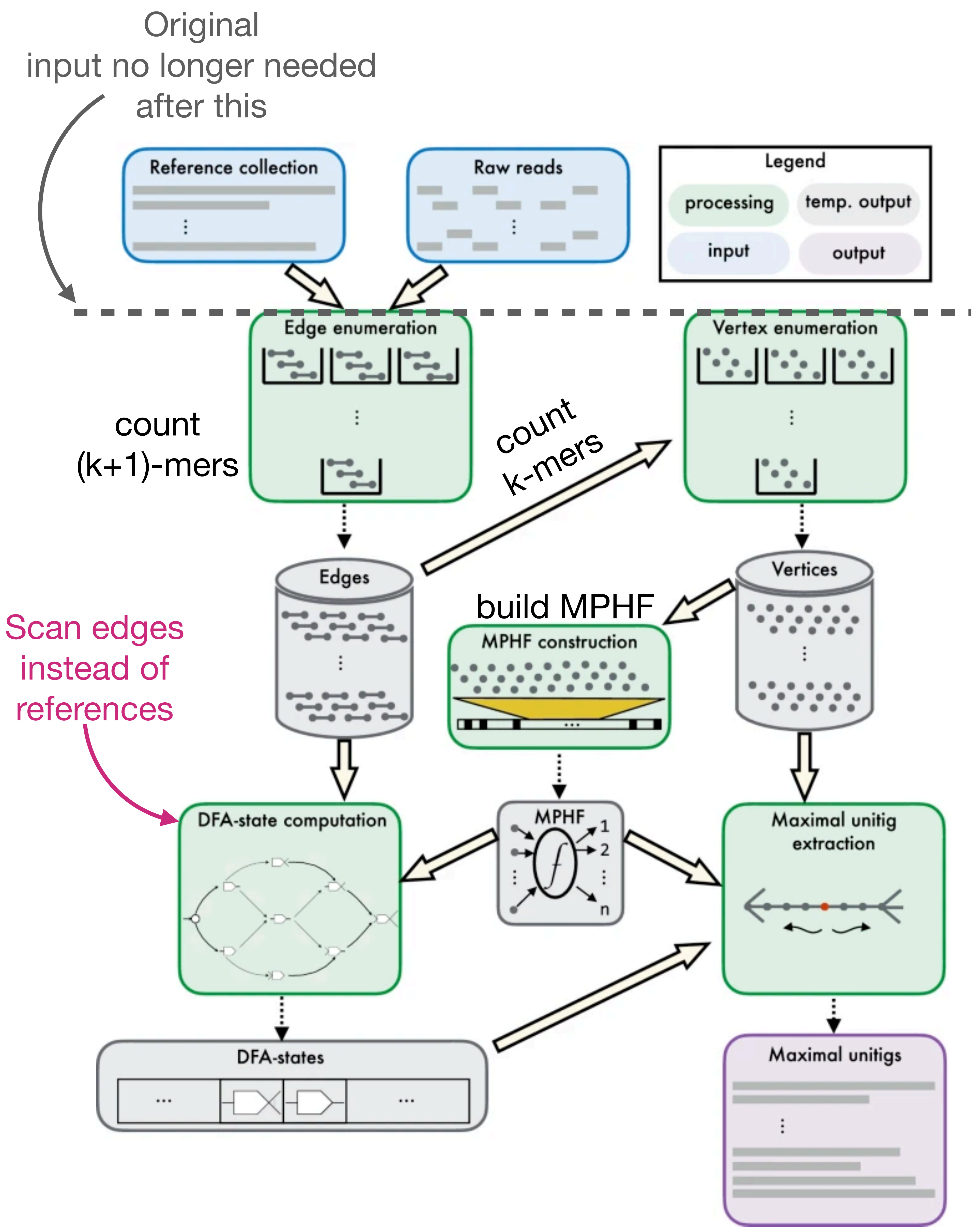
- Bring Cuttlefish 1 methodology to bare on “raw” sequencing data, not just reference collections

- Operate even more efficiently when the input collection is highly-similar / redundant (reducing # of passes over the input data); the “bottleneck” of Cuttlefish 1



Assembling 20 Gbp spruce genome [Birol et al. 2013]

Constructing the DBG (even more) efficiently



Key ideas:

- To construct the compacted DBG using our DFA-based strategy, it is *not* actually necessary to have full input reference sequences.
- Only need to know the DFA nodes (vertices) and to observe their incident subpaths (i.e. their edges)
- Instead of scanning over input references many times, build MPHF of distinct k-mers and scan over distinct (k+1)-mers
- If distinct (k+1)-mers < total sequence size; this approach is much faster!
- Unitigs can be extracted from DFA states directly (state encodes unique predecessor & successor characters for vertices internal to a path)

Constructing the DBG (even more) efficiently

Results constructing from “raw” read data

Table 1 Time- and memory-performance results for constructing compacted de Bruijn graphs from short-read sets

Dataset	*k*	Thread-count	ABySS-Bloom-dBG Small-memory	ABySS-Bloom-dBG Large-memory	Bifrost	deGSM	BCALM 2	CUTTLEFISH 2 Default memory	CUTTLEFISH 2 Match second-best memory	CUTTLEFISH 2 Unrestricted memory
Human	27	8	22 h 18 min (39.3)	20 h 23 min (71.3)	11 h 43 min (48.5)	10 h 36 min (235.8)	04 h 23 min (6.7)	**01 h 13 min (3.2)**	01 h 10 min (6.2)	01 h (11.3)
		16	11 h 38 min (39.3)	11 h 02 min (71.3)	09 h 39 min (48.6)	07 h 08 min (235.8)	04 h 58 min (8.9)	**56 min (3.3)**	56 min (7.6)	51 min (11.3)
	55	8	16 h 32 min (34.0)	15 h 58 min (66.0)	05 h 43 min (43.8)	16 h 50 min (293.2)	04 h 01 min (7.4)	**02 h 20 min (3.5)**	01 h 08 min (7.1)	01 h 03 min (11.3)
		16	09 h 28 min (34.1)	08 h 37 min (66.1)	04 h 16 min (43.9)	15 h 54 min (293.3)	04 h 26 min (10.5)	**02 h 02 min (3.7)**	01 h 11 min (9.5)	51 min (11.3)
Human RNA-seq	27	8	11 h 47 min (33.7)	11 h 22 min (65.7)	06 h 04 min (7.2)	01 h 35 min (87.1)	02 h 58 min (3.8)	**30 min (2.9)**	–	18 min (80.1)
		16	11 h 38 min (39.3)	07 h 38 min (65.7)	07 h 24 min (7.2)	01 h 37 min (87.2)	02 h 46 min (3.9)	**20 min (3.0)**	–	12 min (80.1)
Gut microbiome	27	16	18 h 47 min (42.0)	20 h 12 min (74.0)	03 h 54 min (38.1)	02 h 28 min (157.2)	02 h 34 min (7.7)	**26 min (3.5)**	23 min (6.7)	20 min (26.8)
	55		1 day 17 h 43 min (35.9)	1 day 08 h 09 min (67.8)	02 h 44 min (46.7)	06 h 53 min (293.3)	03 h 02 min (12.5)	**44 min (4.0)**	25 min (11.3)	20 min (69.9)
Soil	27	16	1 d 18 h 35 min (150.4)	14 h 24 min (275.0)	15 h 28 min (274.1)	1 day 14 h 29 min (235.8)	19 h 39 min (52.0)	**02 h 01 min (19.2))**	02 h 18 min (40.9)	01 h 35 min (40.9)
	55		07 h 57 min (128.9)	06 h 36 min (256.8)	05 h 49 min (157.0)	1 day 11 h 05 min (293.3)	08 h 30 min (27.5)	**03 h 02 min (11.1)**	02 h 43 min (23.3)	01 h 38 min (23.3)
White spruce	27	16	*	*X*	*X*	†	2 days 06 h 12 min (36.8)	**10 h 05 min (14.0)**	07 h 47 min (35.2)	07 h 13 min (204.2)
1.14 TB compressed input	55		*	*X*	*X*	†	2 days 09 h 59 min (31.6)	**10 h 12 min (23.8)**	10 h 08 min (31.1)	07 h 24 min (279.3)

*****: insufficient memory   **†**: insufficient disk   X: execution already failed on smaller subset

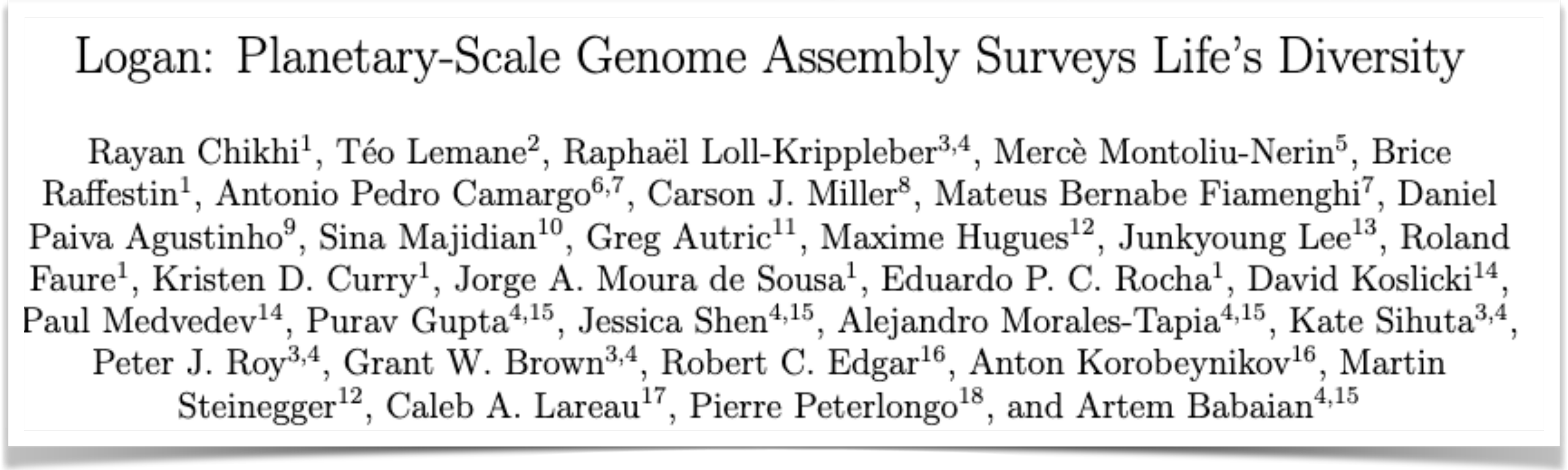
Constructing the DBG (even more) efficiently

Results constructing from "raw" read data

Dataset (genome count)	*k*	Thread-count	Bifrost	deGSM	BCALM 2	Cuttlefish 2 Default memory	Cuttlefish 2 Unrestricted memory
Human gut (30K)	27	8	06 h (155.1)	*Δ*	10 h 06 min (21.5)	**01 h 39 min (15.2)**	01 h 39 min (32.5)
		16	05 h 30 min (155.1)		09 h 05 min (22.0)	**01 h 01 min (15.5)**	59 min (32.5)
	55	8	08 h 47 min (279.2)		11 h 49 min **(18.6)**	**04 h 14 min** (20.6)	03 h 42 min (44.4)
		16	08 h 20 min (279.2)		09 h 45 min **(19.2)**	**03 h 50 min** (20.9)	03 h 10 min (44.3)
Human (100)	27	8	35 h 45 min (355.9)	19 h 23 min (235.8)	♯	**04 h 32 min (27.7)**	04 h 09 min (59.7)
		16	32 h 14 min (355.9)	14 h 07 min (235.8)	♯	**03 h 19 min (28.1)**	02 h 49 min (59.7)
	55	8	*	†	2 days 23 h 31 min (302.9)	**15 h 08 min (56.0)**	13 h 47 min (121.8)
		16	*	†	*	**12 h (56.2)**	11 h 33 min (121.8)
Bacterial archive (661K)	27	16	*X*	*X*	♯	**16 h 38 min (48.7)**	16 h 24 min (104.9)
	55				4 days 10 h 11 min (63.3)	**22 h 44 min (59.9)**	22 h 20 min (129.5)

*****: insufficient memory **≠**: abnormal termination **Δ**: execution stuck at intermediate stage X: execution already failed on smaller subset

Running Cuttlefish 2 on all the things

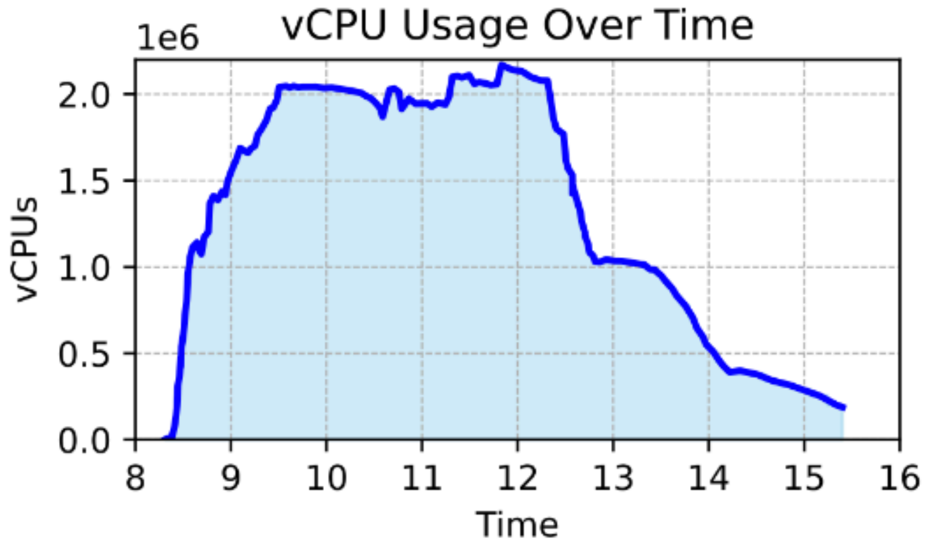


Logan: Planetary-Scale Genome Assembly Surveys Life's Diversity

Rayan Chikhi[1], Téo Lemane[2], Raphaël Loll-Krippleber[3,4], Mercè Montoliu-Nerin[5], Brice Raffestin[1], Antonio Pedro Camargo[6,7], Carson J. Miller[8], Mateus Bernabe Fiamenghi[7], Daniel Paiva Agustinho[9], Sina Majidian[10], Greg Autric[11], Maxime Hugues[12], Junkyoung Lee[13], Roland Faure[1], Kristen D. Curry[1], Jorge A. Moura de Sousa[1], Eduardo P. C. Rocha[1], David Koslicki[14], Paul Medvedev[14], Purav Gupta[4,15], Jessica Shen[4,15], Alejandro Morales-Tapia[4,15], Kate Sihuta[3,4], Peter J. Roy[3,4], Grant W. Brown[3,4], Robert C. Edgar[16], Anton Korobeynikov[16], Martin Steinegger[12], Caleb A. Lareau[17], Pierre Peterlongo[18], and Artem Babaian[4,15]

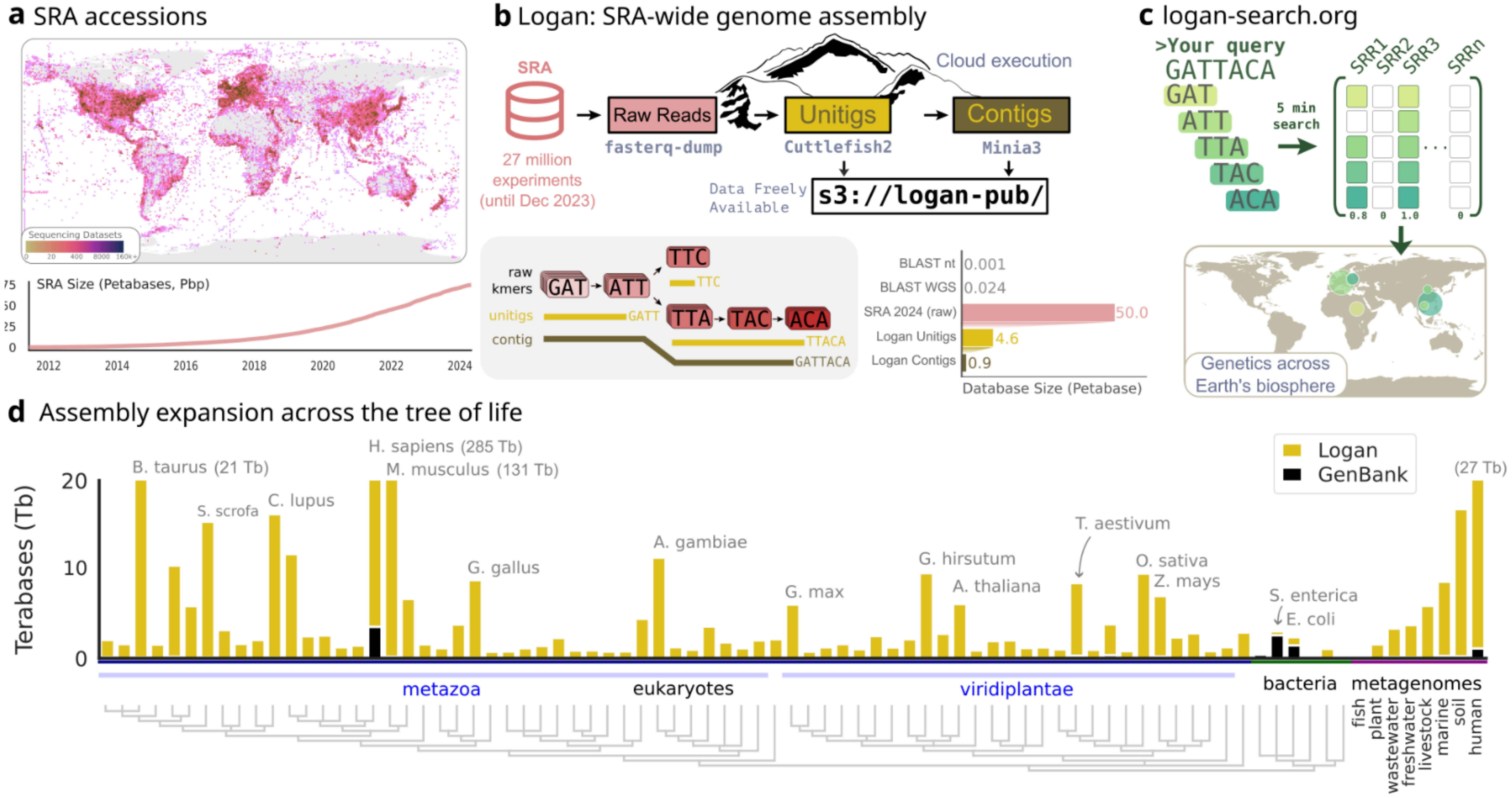
Global statistics

Input SRA Accessions	27 million
Input SRA size	50 petabases
Total CPU Hours	~30 million
Number of Runs	6
Total Runtime	30 hours



Run 6 statistics

Input data	19.6 petabases
Runtime*	7 hours
Peak Number of Instances	73,100
Peak Number of vCPUs	2.18 million
Peak Total EBS storage	52 petabytes



- Cuttlefish2 is an ***enabling technology***!
- Enables assembly & compression *at scale*.
- The `Logan` project is led by Rayan Chikhi (CNRS), author of BCALM & BCALM2; yet chose Cuttlefish for this task.